

Graduation Project Report

***MedAnalyzer: A Python-Based Web Application
for Medical Data Analysis and Risk Prediction
using Flask and Machine Learning.***

Students Names:

Sora Ali Mahasneh. (2021978246)
Tamara Muwaffaq Malkawi. (2021978242)
Amal Ayman Alawadi (2019978009)
Deema Atef Bani Khalid (2020978040)

Supervisor Name:

DR: Shefaa Tawalbeh

Semester: Second 2024/2025

Date: 19th Jun 2025

Students' Property Right Declaration and Anti-Plagiarism Statement

We hereby declare that the work in this graduation project at Yarmouk University is our own except for quotations and summaries which have been duly acknowledged. This work has not been accepted for any degree and is not concurrently submitted for award of other degrees. It is the sole property of Yarmouk University and it is protected under the intellectual property right laws and conventions.

We hereby declare that this report is our own work except from properly referenced quotations and contains no plagiarism.

We have read and understood the school's rules on assessment offences, which are available at Yarmouk University Handbook.

Students:

Name: _____ Signature: _____ Date: _____

Name: _____ Signature: _____ Date: _____

Name: _____ Signature: _____ Date: _____

Name: _____ Signature: _____ Date: _____

Table of Contents

Students' Property Right Declaration and Anti-Plagiarism Statement	i
List of Tables.....	iii
List of Figures	iv
Abstract.....	v
Chapter 1: Introduction.....	1
Chapter 2: Background.....	4
Chapter 3: Design.....	6
Chapter 4: Implementation	16
Chapter 5: Results and Discussion.....	26
Chapter 6: Economical, Ethic, and Contemporary Issues	32
Chapter 7: Project Management.....	35
Chapter 8: Conclusion and Future Work.....	39
APPENDIX A: User Manual	40

List of Tables

<TODO: Provide a list for all tables inside the report (Table number, title, and page). >

Table 1. Table 1: Design Considerations Mapping Table.....14-15

Table 2 : risk management plan.....35

List of Figures

Figure 1: Flowchart for the project.....	6
Figure 2. login page	7
Figure 3. log in page for the patient role.....	7
Figure 4. create account page.....	8
Figure 5. forgot password.....	8
Figure 6. Home page.....	8
Figure 7. Doctors Platform page.....	9
Figure 8. Patient Data Entry Page[1]	9
Figure 9. patient Data Entry Page [2]	9
Figure 10. patient Data Entry Page [3]	10
Figure 11. patient Data Entry Page.[4]	10
Figure 12. Risk prediction route flask [1]	11
Figure 13. Risk prediction route flask [2]	11
Figure 14. Risk prediction route flask [3].....	12
Figure 15. Patient search results by patient.....	26
Figure 16: Patient data result on plot at different times[1].....	27
Figure 17: Patient data result on plot at different times[2].....	27
Figure 18: Patient data result on plot at different times[3].....	28
Figure 19: Prediction Results.....	28
Figure 20: Accuracy Result of predication Model.....	28
Figure 21: The feature importance in predication Model.....	29

Abstract

The MedAnalyzer project is a comprehensive medical data analysis platform developed to streamline the collection, interpretation, and visualization of vital patient health information. The primary purpose of the project is to assist healthcare professionals in managing patient data efficiently and making informed decisions using intelligent analytics. In clinical and research settings, managing patient data manually or across disparate tools can lead to errors, inefficiencies, and delays in diagnosis. MedAnalyzer addresses this problem by providing a unified, user-friendly platform that automates metric calculations, enables time-based tracking, and supports risk prediction.

The solution is a modular full-stack system developed using HTML, CSS, and JavaScript on the front-end, with Python Flask and SQLite on the back-end. Users can input data manually or upload CSV files. The system calculates derived metrics such as BMI, MAP, HRV, and Pulse Pressure, stores data securely, and visualizes trends using Chart.js. A machine learning model is integrated to classify patients into high or low-risk categories. Reports can be downloaded in PDF format using jsPDF. The project was developed over a structured period of several months, involving planning, front-end and back-end development, database integration, and testing phases. Emphasis was placed on user experience, data validation, and performance optimization. Special thanks to all mentors, collaborators, and open-source contributors whose guidance and resources supported the successful completion of MedAnalyzer.

Chapter 1: Introduction

1.1 Problem Statement and Purpose

With the growing reliance on health data and the increasing complexity of medical decision-making, there's a clear need for systems that can analyze patient information quickly and accurately. In many healthcare environments, medical staff still depend on manual data interpretation or subjective judgment, which can lead to delays or even errors in diagnosis—especially in urgent or high-risk situations. This project addresses the lack of a simple, efficient, and intelligent system that combines vital sign data analysis with real-time risk prediction. The goal is to build a smart, user-friendly tool that helps healthcare professionals (or even regular users) make better decisions by providing instant insights based on accurate data interpretation.

1.2 Background

Recent advances in machine learning and data analytics have opened up opportunities for smarter healthcare solutions. Many models have been developed to predict patient outcomes, but most remain stuck in research labs, with little practical use in real-world environments. The main issues with existing solutions include lack of integration between data entry, analysis, and prediction; high technical complexity, making tools hard to use; and no clear or interactive visual feedback to support quick understanding. These gaps highlight the need for an end-to-end system that combines automation, usability, and clarity.

1.3 Aims and Objectives

The main aim of this project is to develop a complete and usable system for analyzing patient data and predicting risk levels in real time. The objectives include designing a backend system that supports real-time data handling, building a clean and easy-to-use frontend for inputting patient data and viewing results, and integrating a trained machine learning model to make risk predictions (e.g., High or Low). Additionally, the system should visualize results using clear, interactive charts, generate downloadable reports in PDF format summarizing the analysis, and be evaluated using realistic datasets and usability testing.

1.4 How Has the Problem Been Solved till Now?

Several approaches have attempted to address this issue in the past. Manual systems involve collecting and interpreting data without any real-time analysis or predictive capabilities. Standalone models are

often developed for academic purposes only, lacking user interfaces or real-world deployment. Generic cloud tools may offer partial analytics but typically do not support healthcare-specific features or workflows. These methods often lack practicality, integration, or ease of use—especially in fast-paced healthcare environments.

1.5 Main Solution Idea

This project proposes a full-stack system called MedAnalyzer that allows users to enter patient data through a web interface, instantly analyze vital signs such as heart rate, blood pressure, temperature, and oxygen level, and receive real-time risk predictions using a machine learning model. Users can also view dynamic charts and trends and download a PDF report of the patient’s analysis. The idea is to offer everything in one place—data input, smart predictions, clear visuals, and ready-to-use results.

1.6 Key Technical Details and Evaluation

The technologies used in this project include Flask (Python) for the backend, which handles API requests, connects to the database, and runs the machine learning model. SQLite is used as the database to store patient records and analysis history. For machine learning, a pre-trained model such as Random Forest or XGBoost is used to predict the risk level based on input data. The frontend is built using HTML, CSS, and JavaScript, with Chart.js used for visualizations. PDF reports are generated using jsPDF. The system was evaluated using both synthetic and real-world datasets, with key performance indicators such as accuracy, speed, and user experience guiding the evaluation. Usability testing confirmed that the interface is intuitive and responsive.

1.7 Contributions

This project offers several important contributions:

- A fully integrated web-based tool for health data analysis and real-time prediction.
- A seamless connection between data input, machine learning inference, and visual output.
- A user interface that works for both technical and non-technical users.
- A simple report generation system for printing or saving patient results.
- A reusable architecture that could be adapted to other domains or extended in the future.

1.8 High-Level System Diagram

The main components of the MedAnalyzer system include the user interface, where users input patient data and view the results, and the backend powered by a Flask server, which processes the data and communicates with both the database and the machine learning model. The database, built with SQLite, stores all patient data for analysis, history tracking, and user information. The machine learning model receives the input and returns a risk prediction. Additionally, the visualization and report generator display the results through dynamic charts and enables users to download a PDF report. The system flow follows a straightforward process: the user inputs data → the backend processes it → the machine learning model makes a prediction → the results are displayed on the dashboard → the user can download a PDF report.

1.9 Summary of Report Structure

The rest of this report is structured as follows:

Chapter 2: Literature Review

A detailed look at existing systems, technologies, and research related to health data analytics.

Chapter 3: System Design

Describes how the system works, the architecture, and how components interact.

Chapter 4: Implementation

Covers the development process, tools used, and any technical challenges faced.

Chapter 5: Evaluation

Presents the results of system testing and analysis of its performance.

Chapter 6: Conclusion and Future Work

Summarizes the project outcomes and suggests improvements or future development directions.

Chapter 2: Background

2.1 Background of the Problem

In recent years, the healthcare industry has seen a growing need for systems that can support better decision-making based on real-time patient data. Vital signs like heart rate, blood pressure, oxygen saturation, and body temperature play a crucial role in understanding a patient's health status. However, interpreting these values accurately and quickly remains a challenge, especially in environments with limited resources or overloaded medical staff.

Studies in medical informatics highlight that while many healthcare institutions collect large volumes of patient data, this data is often underused due to a lack of smart tools that can process and analyse it effectively. Traditional systems mostly rely on manual interpretation, which is not only time-consuming but prone to human error.

2.2 Importance and Context of the Problem

Real-time monitoring and interpretation of patient vitals is essential in various medical settings, including emergency rooms, intensive care units (ICUs), outpatient clinics, and even home care. Delayed or missed warning signs can result in worsening health conditions or even preventable deaths. With the increasing availability of wearable sensors and electronic health records, there's now an opportunity to shift from passive data collection to active data interpretation. The importance of this shift is especially apparent in chronic disease management, elderly care, and remote patient monitoring. Thus, building systems that can predict risk levels and assist in early detection can significantly improve patient outcomes and reduce pressure on healthcare systems.

2.3 Target Market and Their Needs

The primary target users of such systems include healthcare professionals like doctors, nurses, and emergency responders who need fast and accurate insights to make clinical decisions. Hospitals and clinics also benefit as institutions seeking to improve efficiency, reduce workload, and enhance the quality of patient care. Home care providers, who monitor elderly or chronically ill patients remotely, are another key user group. Additionally, patients and their families who want to track health and receive alerts when readings are abnormal are important users. Their main needs include easy-to-use interfaces, real-time feedback, clear visualizations and alerts, and reliable predictions without requiring deep technical knowledge.

2.4 Ethical and Environmental Considerations

Ethical Concerns include data privacy, as patient data is highly sensitive and any system handling such information must comply with regulations like HIPAA (in the U.S.) or GDPR (in Europe), ensuring encryption and secure storage. Bias in machine learning is another important issue; predictive models must be trained on diverse datasets to avoid biased results that could negatively impact certain patient groups. Additionally, there is a risk of over-reliance on AI, so medical professionals should be encouraged to treat the system as a decision support tool—not as a replacement for their medical judgment. Regarding environmental considerations, since this is a software-based system running on existing infrastructure such as cloud servers or local hospital systems, it has minimal environmental impact. However, promoting efficient coding practices and using low-power computing environments can help minimize energy usage.

2.5 Summary of Existing Approaches

Several methods have been used in the past to analyze patient data and predict health risks. Manual interpretation involves doctors and nurses manually reviewing patient vitals, which, while effective in many cases, is slow and subject to fatigue or oversight. Rule-based systems use predefined thresholds (e.g., alert if heart rate > 120 bpm), but these lack adaptability and can trigger false alarms. Statistical models like linear regression and logistic regression have been applied for basic risk scoring; although transparent and interpretable, they often fail to capture complex patterns in patient data. Machine learning models, including Decision Trees, Random Forest, Support Vector Machines, and more recently Deep Learning models, have shown strong potential, with studies demonstrating improved prediction accuracy, especially in ICU patient monitoring. However, many of these models remain confined to research papers and are not integrated into usable interfaces. Mobile and cloud-based platforms collect data from wearables and provide basic insights, but they are limited by a lack of clinical-grade accuracy, limited customizability, and insufficient transparency regarding how predictions are made.

Chapter 3: Design

Flowchart for the project:

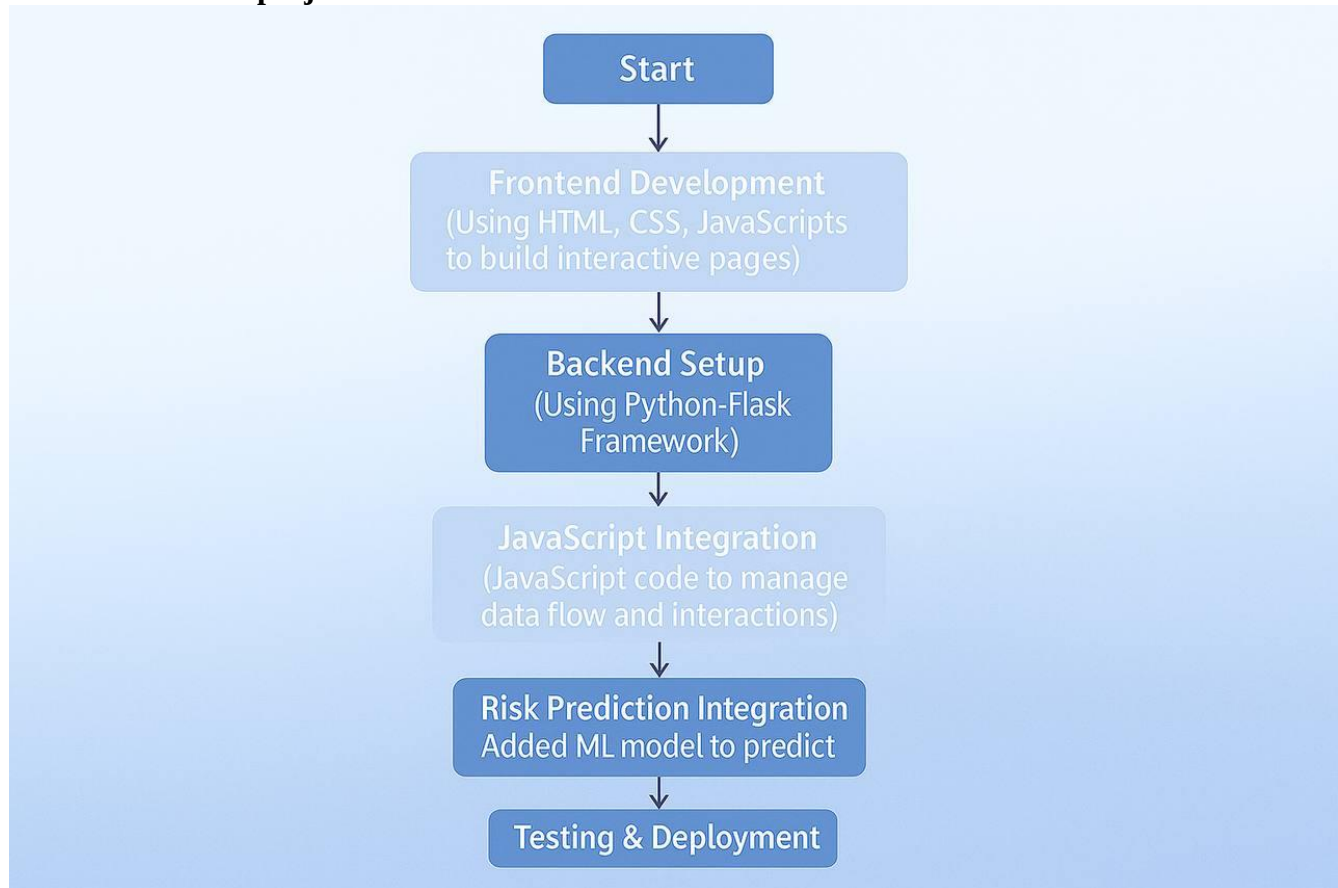


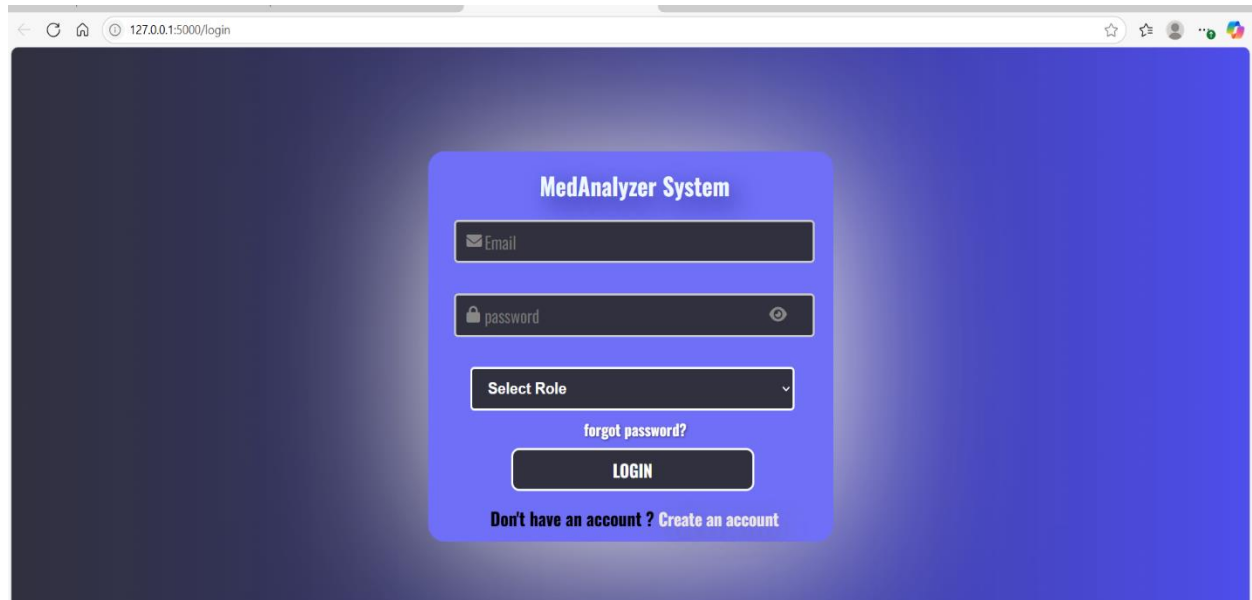
Figure 1: Flowchart for the project.

3.1 Frontend Development

The development of the MedAnalyzer system began with designing the user interface using core web technologies such as HTML, CSS, and JavaScript. The goal was to build an interactive and user-friendly experience that includes all the essential pages required for the system's operation. A **Login** page was implemented to allow access for doctors, nurses, and patients based on their assigned roles. Additionally, a **Create Account** page was developed for new users to register, alongside a **Forgot Password** page for recovering login credentials.

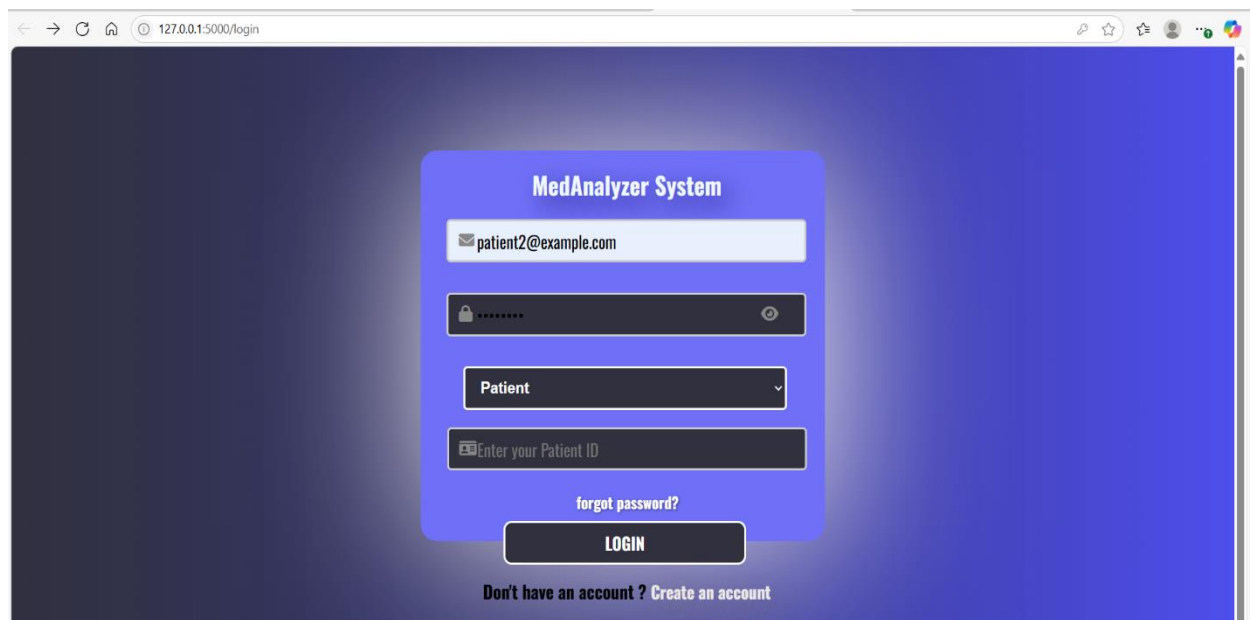
For data collection, a **Data Entry** page was created with a manual input form for entering vital signs, as well as the option to upload CSV and Excel files containing bulk patient data. An **Analytics Page** was also developed to display time-based visualizations of patient data and provide a real-time risk prediction (High/Low Risk) using a machine learning model. Physicians are able to write medical prescriptions and generate a downloadable PDF report that includes all visualizations and notes.

Moreover, a **Dashboard Page** was developed for doctors to access a comprehensive view of patient history, medical readings, and current medications. The system also includes a **Doctors Info Page** where profiles are displayed with names, emails, specialties, and years of experience.



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000/login". The page has a dark blue background. In the center, there is a light blue rounded rectangle containing the login form. The form is titled "MedAnalyzer System" in bold. It includes an "Email" input field, a "password" input field with a toggle icon, a "Select Role" dropdown menu, a "forgot password?" link, a "LOGIN" button, and a link "Don't have an account ? Create an account" at the bottom.

Figure 2: login Page.



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000/login". The page has a dark blue background. In the center, there is a light blue rounded rectangle containing the login form. The form is titled "MedAnalyzer System" in bold. It includes an email input field with the text "patient2@example.com", a password input field with a toggle icon, a "Patient" dropdown menu, an "Enter your Patient ID" input field, a "forgot password?" link, a "LOGIN" button, and a link "Don't have an account ? Create an account" at the bottom.

Figure 3: Login Page for Patient.

Create Your Account

Please fill out the information below to create an account for accessing the MedAnalyzer system.

Email:
Enter your email

Password:
Enter your password

Confirm Password:
Confirm your password

Role:
Doctor

Create Account

[Back To Login](#)

Figure 4: Create Account page.

Reset Your Password

Enter your email

Enter new password

Reset Password

[Remembered your password? Login here](#)

Figure 5: forgot password Page.

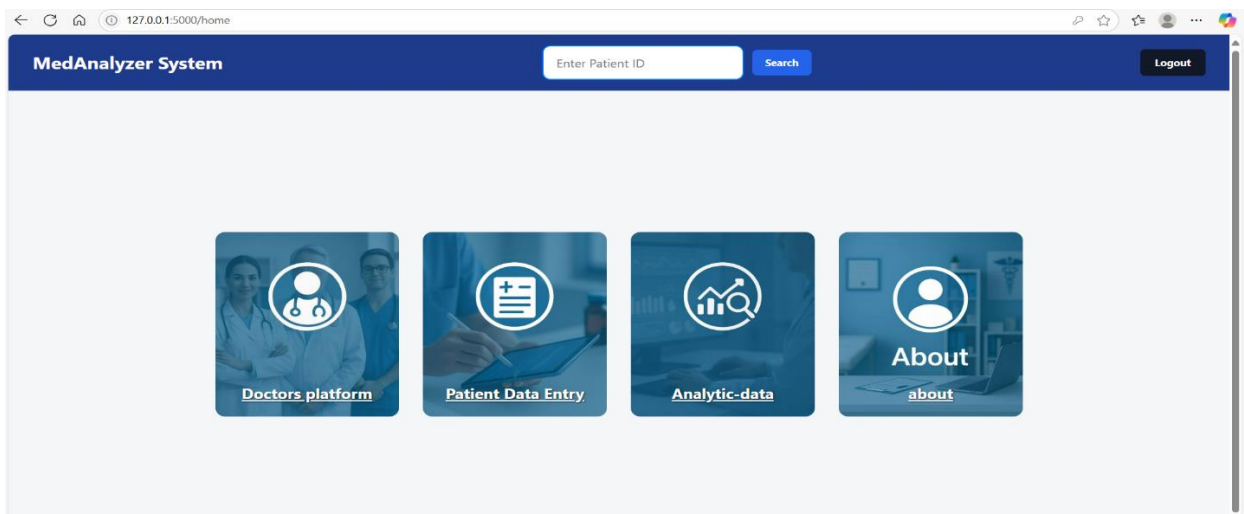


Figure 6: Home Page.

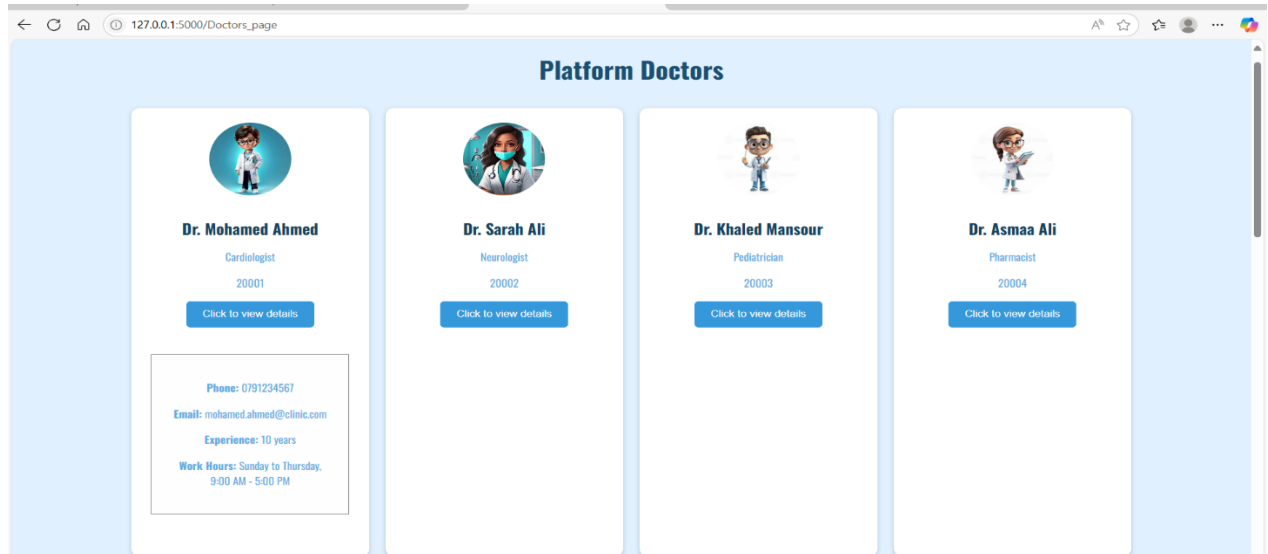


Figure 7: Doctors platform page.

The screenshot shows a web application titled "Enter Patient Data". It contains a central form with several input fields. The fields are labeled: "Patient ID :", "Heart Rate :", "Respiratory Rate :", "Timestamp :", "Body Temperature :", "Oxygen Saturation :", and "Systolic Blood Pressure :". The "Timestamp" field has a placeholder "mm / dd / yyyy -- : -- : --" and a calendar icon. The "Respiratory Rate" field has a dropdown arrow. The form is set against a light gray background.

Figure 8: Patient Data entry page [1].

This screenshot shows the continuation of the "Enter Patient Data" form. It includes input fields for "Diastolic Blood Pressure :", "Age :", "Gender :", "Weight (kg) :", and "Height (m) :". The "Gender" field is a dropdown menu with "Select Gender" as the current selection. Below these fields are four blue buttons: "Calculate BMI", "Calculate Pulse Pressure", "Calculate MAP", and "Calculate HRV". At the bottom, there are two more input fields labeled "HRV :" and "Pulse Pressure :".

Figure 9: Patient Data entry page [2].

Figure 10: Patient Data entry page [3].

Figure 11: Patient Data entry page [4].

3.2 Backend Setup

The backend of the system was implemented using the Flask framework in Python to serve as a bridge between the frontend and the database. Various routes were set up to handle operations like saving data, uploading files, retrieving analytics, and running predictions. Among the key routes are `/save_patient` and `/upload-csv`, responsible for processing manual entries and file uploads, respectively, and `/Analytics` to retrieve patient data and perform analysis.

A pre-trained machine learning model using XGBoost was integrated to predict patient risk based on vital signs. Patient data is passed to the model, which returns a classification ("High" or "Low Risk"), along with a visualization of feature importance to support doctors in interpreting the results. The backend also manages user roles, access control, and data validation, all implemented with strong security practices.

```

687 model = xgb.XGBClassifier()
688 model.load_model('final_xgb_model (4).json')
689 scaler = joblib.load('scaler (4).pkl')
690
691 @app.route('/predict_risk', methods=['POST'])
692 @login_required()
693 def predict_risk():
694     patient_id = None
695
696     if session['role'] == 'patient':
697         patient_id = session.get('PatientID')
698     else:
699         data = request.get_json(silent=True)
700         patient_id = data.get('patient_id') if data else None
701
702     if not patient_id:
703         return jsonify({'error': 'Patient ID is required'}), 400
704
705     try:
706         with sqlite3.connect("patient_entry.db") as conn:
707             cursor = conn.cursor()
708             cursor.execute('''
709                 SELECT
710                     HeartRate, RespiratoryRate, BodyTemperature,
711                     OxygenSaturation, SystolicBloodPressure, DiastolicBloodPressure,
712                     Age, Gender, Derived_BMI
713                 FROM patients
714                 WHERE PatientID = ?
715                 ORDER BY Timestamp DESC
716                 LIMIT 1
717             ''', (patient_id,))
718             row = cursor.fetchone()

```

Figure 12: risk prediction route flask [1].

```

719         if not row:
720             return jsonify({'error': 'No data found for this patient'}), 404
721
722         gender_value = row[7]
723         gender_num = 1 if str(gender_value).lower() == 'male' else 0
724
725         model_input = {
726             "HeartRate": float(row[0]),
727             "RespiratoryRate": float(row[1]),
728             "BodyTemperature": float(row[2]),
729             "OxygenSaturation": float(row[3]),
730             "SystolicBloodPressure": float(row[4]),
731             "DiastolicBloodPressure": float(row[5]),
732             "Age": float(row[6]),
733             "Gender": gender_num,
734             "Derived_BMI": float(row[8])
735         }
736
737         input_df = pd.DataFrame([model_input])
738
739         column_mapping = {
740             "HeartRate": "Heart Rate",
741             "RespiratoryRate": "Respiratory Rate",
742             "BodyTemperature": "Body Temperature",
743             "OxygenSaturation": "Oxygen Saturation",
744             "SystolicBloodPressure": "Systolic Blood Pressure",
745             "DiastolicBloodPressure": "Diastolic Blood Pressure",

```

Figure 13: risk prediction route flask [12].

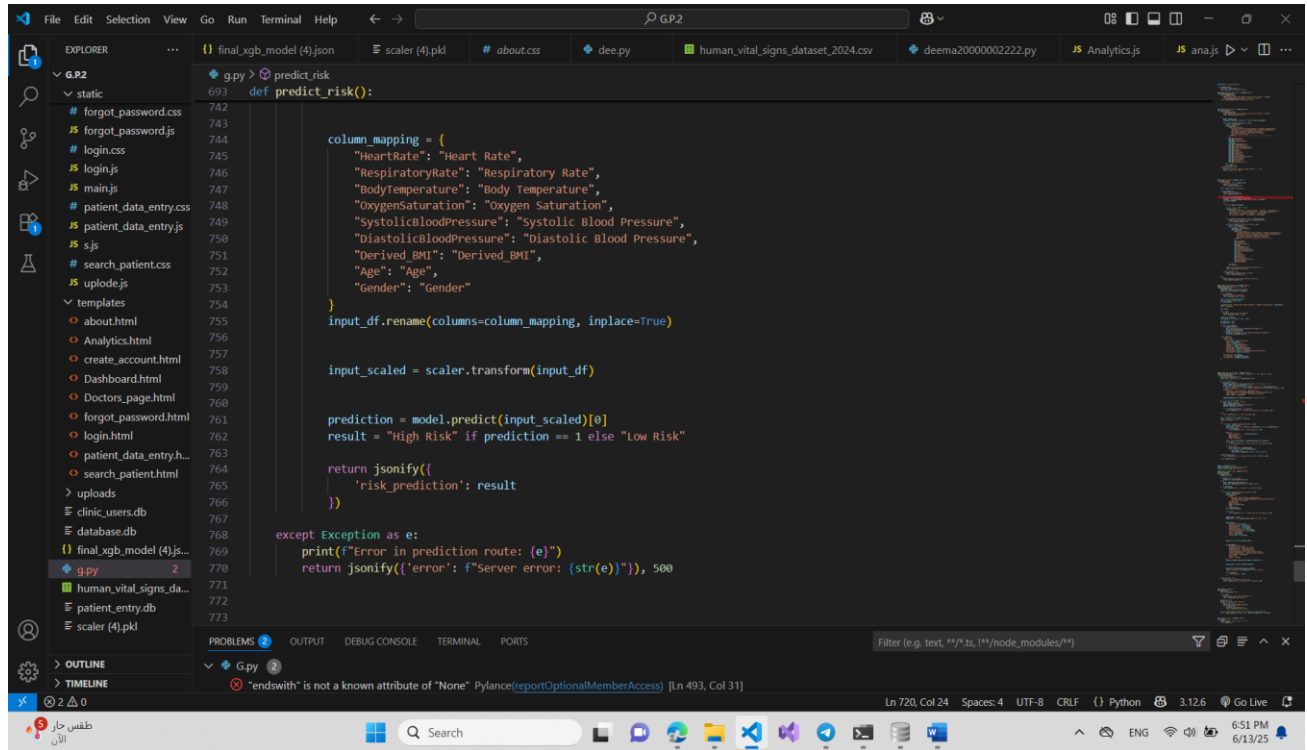


Figure 14: risk prediction route flask [3].

3.3 Database Creation

The system uses SQLite for data storage due to its simplicity and seamless integration with Flask in a local environment. Two **separate databases** were created:

- The first handles **clinical user accounts**, with a users table storing information such as name, email, hashed password, role (doctor, nurse, patient), specialty, and years of experience.
- The second stores **patient vital data**, using a patients table that includes both original and derived values like HRV, MAP, BMI, and Pulse Pressure, along with a timestamp for each entry.

Separating the two databases improved system organization and enhanced data security by isolating login credentials from clinical records, thus supporting compliance with standards like GDPR and HIPAA.

3.4 Route Implementation

All Flask routes were implemented to connect the frontend with backend logic and databases. These routes include /login for user authentication, /register for account creation, /forgot-password for password recovery, as well as /data-entry and /upload-csv for handling vital data input. Additional

routes were built for retrieving patient analytics, displaying doctor profiles, and filtering patient history and medication details based on roles and permissions.

Each route verifies the user's identity and role before processing any requests, ensuring patients cannot view or modify other patients' data and that doctors have the proper access to write prescriptions and view analytical reports.

3.5 JavaScript Integration

JavaScript was integrated throughout the MedAnalyzer frontend to enable interactivity and dynamic updates. It handles form submissions, fetches backend data, and manages user navigation. The **Chart.js** library was used to create interactive and time-based charts for vital signs, while **jsPDF** was employed to allow physicians to export full medical reports, including analytics and handwritten notes, as downloadable PDF files.

3.6 Risk Prediction Integration

In designing the intelligent risk prediction model, our objective was to accurately classify patients into either *High Risk* or *Low Risk* categories based on a set of vital physiological parameters. The design process began by identifying the most relevant features from the dataset, such as heart rate, respiratory rate, body temperature, oxygen saturation, systolic and diastolic blood pressure, age, and gender. Derived or redundant features like weight, height, MAP, HRV, and pulse pressure were excluded to improve model clarity and performance.

The XGBoost classifier was selected for the prediction task due to its high accuracy and robustness when working with structured data like medical records. It also supports built-in regularization, efficient training, and handling of missing values, making it a suitable choice for a clinical application where precision is critical.

Feature engineering played a vital role in shaping the model's performance. Gender was encoded numerically (0 for male, 1 for female) and the target variable, Risk Category, was mapped to 0 for Low Risk and 1 for High Risk. All features were standardized using `StandardScaler`, ensuring they had zero mean and unit variance. This normalization step was crucial to ensure the learning algorithm could converge efficiently and fairly across all features.

Furthermore, to guarantee that the model generalizes well to new data, we incorporated Stratified 5-Fold Cross-Validation into the design. This approach ensured that both classes were proportionally represented in each training and testing split, providing a reliable estimate of model performance.

Finally, an important part of the design involved planning for interpretability. Feature importance analysis was integrated as a step to visually and quantitatively assess which physiological indicators most influenced the model's decisions. This design choice improves trust and understanding of the system by medical professionals.

3.7 Testing and Deployment

After completing development, the system underwent extensive functional and integration testing to ensure all components worked seamlessly. We tested form inputs, file uploads, data analysis, prediction accuracy, PDF generation, and user role restrictions. Once confirmed stable, the system was deployed locally and prepared for future deployment on a cloud platform to enable scalability.

3.8 Design considerations table.

Design consideration	Project application	Relevant location in report
Performance	The system displays patient data rapidly and performs health risk predictions instantly.	Section 3.6
serviceability	The system is designed to allow healthcare providers to easily access and interpret patient data and update reports when needed.	Section 3.4 Section 3.5
Economic	The system is web-based using open-source technologies, reducing overall development and maintenance costs.	Section 3.2 Section 3.3
Environmental	The system is fully digital and minimizes the need for printed records, helping reduce paper waste.	Section 3.1 Section 3.7
Environmental Sustainability	By digitizing health data and reducing manual paperwork, the system contributes to	Section 3.3 Section 3.7

	sustainable healthcare practices.	
Manufacturability	The project is software-based and can be easily deployed and scaled using cloud services without physical manufacturing	Section 3.7
Ethical	The system ensures that each patient can only view their own data, maintaining privacy and confidentiality.	Section 3.7
Health and safety	Enables better monitoring of patient health indicators, supports doctors in making timely decisions, and includes secure access	Section 3.2
Social	Improves healthcare delivery, strengthens communication between doctors, patients, and nurses.	Section 3.2
Political	Complies with healthcare regulations and standards, making it suitable for adoption by institutions in various countries.	Section 3.2

Table 1: Design Considerations Mapping Table.

Chapter 4: Implementation

4.1 Tools and methods

In this project, we developed a medical web application to support doctors, nurses and patients in analyzing health data and generating medical insights. The implementation involved the use of several tools and technologies, categorized as follows:

4.1.1 Programming Language

- **Python (core backend logic and Primary Language for Data Analysis):** We used Python because it is powerful and easy to work with. It served as the main language for writing all the backend logic of the website. Through Python, we were able to read patient data from CSV files, analyze it using artificial intelligence models, interact with databases, and send the results to the frontend. It was essentially the backbone of all the smart operations happening on the website.[2][3]

4.1.2 Framework

- **Flask:** is the framework we used to build the website. It is lightweight and easy to use. We used it to connect the web pages with the code written in Python. For example, when a user uploads a CSV file, Flask receives the file, passes it to the backend code for processing, and then returns the results to the user. We also used it to organize the routes such as /login, /dashboard, /analyze, and others.[1][16]

4.1.3 Frontend Technologies

- **HTML5(HyperTextMarkup Language):** is the standard language used to create and design the structure of web pages. It tells the web browser how to display text, images, links, buttons, and other content on a website. In our project, we used HTML5 to build the structure of the website pages. It defined where each element should appear, such as buttons, input fields, tables, and charts. For example, the login page included a form for user credentials, the file upload page provided a section to upload patient data in CSV format, and the analysis page displayed statistical results and graphs using well-organized HTML components.[12]
- **CSS3(Cascading Style Sheets):** is a formatting language used to define the appearance and design of web pages, it complements HTML (which handles content and structure) by adding colors, fonts, layouts, effects, and responsiveness to websites.

We used CSS to style the website and make the interface user-friendly and visually clear. It controlled the layout, colors, font sizes, and alignment of elements across the pages. CSS made sure that buttons were properly styled, and the overall look of the system was clean and professional.[12]

- **JavaScript:** is a fundamental programming language that adds dynamism and interactivity to websites. Without it, web pages remain static and unchangeable, much like a traditional paper catalog. JavaScript added interactivity to the website. It allowed us to dynamically display patient data, generate PDF reports using the jsPDF library, and render visual charts through Chart.js. JavaScript also handled user actions such as button clicks, showing or hiding sections based on selected filters, and fetching analysis results from the backend without refreshing the page. This helped create a smooth and responsive user experience.[12]

4.1.4 Libraries

- **Pandas, NumPy:** In our project, we used Pandas and NumPy for data processing. After uploading the CSV file, Pandas was used to read and structure the data in a DataFrame, making it easier to analyze and clean. NumPy was then used to perform statistical operations such as calculating the mean, standard deviation, and identifying abnormal values. Together, these libraries helped prepare the data for machine learning and visualization.[4][5]
- **Matplotlib, Seaborn:** For data visualization, we used Matplotlib and Seaborn during the model development phase. These libraries helped us create visual plots to explore relationships between variables and understand trends in the dataset. They were especially useful in validating the performance of our model and presenting data insights clearly.
- **Joblib, Pickle:** To handle machine learning models, we used Joblib and Pickle to load pre-trained models. Once the model was trained and saved, it could be reloaded whenever the system started, allowing us to make predictions on patient data without retraining the model every time.
- **Scikit-learn:** We used Scikit-learn to train our machine learning model. Specifically, we implemented a Random Forest Classifier due to its high accuracy and reliability in classifying medical conditions based on patient data. Once trained, the model was exported and integrated into our backend for real-time predictions.

- **Flask-CORS, werkzeug, session handling:** For full-stack functionality, we used Flask-CORS to enable communication between the frontend and backend from different origins. Werkzeug supported file handling and request processing, especially for file uploads. We also used Flask sessions to manage user authentication and maintain login states across pages.
- **SQLite3:** A built-in database library in Python for using SQLite databases (lightweight, file-based DB). Used to manage user accounts and patient medical records by creating and modifying .db files like clinic_users.db and patient_entry.db.
- **jsPDF:** jsPDF is a JavaScript library that enables the generation of PDF documents directly from the browser. We used jsPDF to generate downloadable PDF reports for each patient. These reports include the patient's ID, selected date, doctor's notes, and statistical summaries (average, minimum, maximum, and standard deviation) for each selected measurement. This allows medical staff to easily save and share patient data in a clean, printable format.[10]
- **Chart.js:** is a popular JavaScript library used for creating responsive, interactive charts using HTML5 canvas. We used Chart.js to visualize patient health data through dynamic line charts. The chart displays selected medical measurements (such as heart rate, blood pressure, etc.) over time, allowing doctors to clearly track trends and patterns. The chart updates automatically based on the chosen patient ID, timestamp, and selected metrics.[9]

4.1.5 Database

- **SQLite Databases:** In our project, we used SQLite as the database system because it is lightweight and easy to integrate, making it ideal for small-scale applications. We worked with two main database files:
 - **clinic_users.db:** file was used to store user credentials, specifically for doctors. It contained information such as email addresses and passwords. This database played a key role in the login process, where user credentials were verified before granting access to the internal pages of the system like file upload and data analysis.[8]

- **patient_entry.db:** file was responsible for storing patient data. After uploading a CSV file, the system extracted medical data such as patient ID, heart rate, body temperature, oxygen saturation, blood pressure, and timestamps, and saved them into this database. This allowed the backend to later retrieve and analyze the data, generate visual charts, and produce downloadable reports for each patient.

4.1.6 Machine Learning Frameworks:

The implementation phase started with training the final model using the preprocessed dataset. After validating the model's performance through cross-validation, we retrained it on the entire cleaned dataset and saved it in JSON format (final_xgb_model (4).json). Along with the model, we also saved the fitted scaler (scaler (4).pkl) and the feature names for consistent preprocessing during deployment.

To enable real-time predictions in a clinical setting, we integrated the model into a Flask web application. A dedicated RESTful API endpoint, /predict_risk, was developed to receive POST requests. This endpoint accepts a PatientID, retrieves the most recent physiological data from the SQLite database, encodes and scales the features, and then uses the trained XGBoost model to make a prediction. The predicted risk level—either "High Risk" or "Low Risk"—is returned as a JSON response.

Security and privacy were key considerations in implementation. The system ensures that only authenticated users can access the prediction service, and patients can only retrieve their own data. These measures align with ethical design principles and healthcare data regulations.

On the front end, the model's results are integrated into the dashboard where users can select a patient and view their predicted risk level in real-time. Visualization tools such as Chart.js are used to display vital sign trends over time, while the prediction outcome is presented alongside a horizontal bar chart of feature importances, providing additional interpretability.

For documentation and reporting, we implemented a PDF export feature using jsPDF. Doctors and nurses can generate downloadable reports containing patient vitals, the model's prediction, and the interpreted significance of each feature. This feature improves communication among healthcare providers and facilitates efficient documentation of clinical decisions.[6][7][17]

5.1.7 Other Tools

- **CSV Uploading:** enabled for batch input of patient data.
- **Visual Studio Code (VS Code):** Visual Studio Code (VS Code) is a powerful tool and source code editor that we use to write and edit code easily and efficiently. It supports many programming languages. We used Visual Studio Code as the main code editor throughout the project for both front-end and back-end development. The editor was easy to use. Through it, we wrote and designed HTML, CSS, and JavaScript files for the front-end, and also developed the server-side and back-end programming using python language smoothly.[11]
- **Data Processing:** Preprocessing involved standardization (via scaler.pkl) and encoding categorical variables (label_encoder.pkl).
- **Version Control & File Structure:** The codebase is modular and organized, with folders for templates, static resources, and ML models.
- **Model Serialization:** Trained models were saved using joblib and pickle for later inference during real-time web interactions.
- **Model and Preprocessing Files:** We used files saved in .pkl and .json formats to store the trained models (such as the Random Forest and XGBoost models), in addition to preprocessing tools like scaler.pkl for value normalization and label_encoder.pkl for categorical encoding.

4.2 Infrastructure Dependencies

The project operates on lightweight, local infrastructure with the following dependencies:

- **Web Server:** The website runs using the built-in server in the Flask framework, which is suitable for local development. This means that all operations and processing happen inside the user's or developer's machine environment, without needing to deploy the site on the internet.
- **Browser:** for interacting with the user interface.
- **Python packages:** Flask, pandas, scikit-learn, xgboost, joblib, pickle, numpy.

- **File Storage:** An "uploads" folder inside the project is used to store the CSV files uploaded by users. These files contain patient data and are analyzed immediately after being uploaded through the interface.
- **Model Hosting:** All machine learning models, such as `model.pkl` and `random_forest_model.pkl`, are stored locally as `.pkl` files and loaded at runtime using libraries like `joblib` and `pickle`. This allows predictions to be made without calling external servers or APIs.
- **No Cloud Services:** The project does not rely on any external cloud services like Google Cloud or AWS. All processes, from data uploading to processing and report generation, are done locally. This enhances privacy, reduces costs, and simplifies the system.

4.3 Design and Implementation Trade-offs

During the design and implementation of the project, we faced several decisions where we had to make trade-offs between efficiency, time, usability, and system complexity. The most significant of these trade-offs include:

- **Operating locally only:** We chose to run the system entirely on a local machine without using any cloud services. This decision simplified development and reduced complexity, but it limited the project's scalability and ability to be accessed from multiple devices or locations.
- **Using SQLite as the database:** We selected SQLite because it is lightweight, easy to use, and requires no complex setup. However, it's not ideal for large-scale systems or handling multiple concurrent users in the future.
- **Choosing Flask instead of Django:** We went with Flask for its simplicity and flexibility, which gave us more control over the project structure. On the downside, we had to manually implement features like session handling and user authentication, which are built-in in frameworks like Django.
- **Loading pre-trained ML models from files:** We trained the machine learning models separately and stored them as `.pkl` files to be loaded at runtime. This approach made the system faster and easier to run, but it prevented the models from learning continuously from new data within the application.
- **Building the front-end manually with HTML, CSS, and JavaScript:** We opted to design the user interface manually without relying on frameworks like React or Bootstrap to keep the UI lightweight and fast. However, this required more time and effort to create a polished and user-friendly design.

- **Using two separate databases:** We maintained one database for user credentials and another for patient data analysis. While this added some organization, it also increased the complexity of managing and linking data between them.
- **No automated tests:** We didn't do automated software testing of the code, which could cause problems later if we want to make changes to the system and ensure that everything works after the changes.
- **Sticking with classic models:** We used models like Random Forest and XGBoost and didn't try neural networks or deep models, to save time and keep the system simple and fast, but we may have lost higher accuracy in some cases.

4.4 Implementation Dependencies & Assumptions

During the implementation of this project, several tools and environmental requirements were necessary for the system to function properly. Additionally, some assumptions were made to ensure smooth operation. The following points summarize both dependencies and assumptions:

4.4.1 Dependencies:

- A properly installed Python environment, using a version compatible with all required libraries.
- Installation of essential packages using pip, such as: Flask and Flask-CORS for running the local web server and connecting the frontend to the backend, Pandas and NumPy for processing patient data, Matplotlib and Seaborn for generating visual charts, Scikit-learn for machine learning model training, joblib or pickle for loading pre-trained models, Werkzeug for secure session handling and backend utilities.
- Availability of the two databases: database.db and clinic_users.db, both containing the correct schema and tables.
- Presence of an uploads/ folder to store the uploaded CSV files.
- Machine learning models should be pre-trained and saved as .pkl files inside the project directory.
- The project relies on a specific directory structure including uploads/, templates/, static/, and pre-trained model files (.pkl, .json) located in the root folder.

- The system uses Matplotlib and Seaborn for generating charts on the server side. The rendering backend is set to 'Agg' to support headless environments.
- A local database (clinic_users.db) is initialized if not already present. It must contain a users table with fields: email, password, and role.
- The uploads/ folder must exist for storing patient data files. It is automatically created if missing.

4.4.2 Assumptions:

- **Correct Format of Input Files:** It is assumed that the user uploads patient data in the form of CSV files, with properly formatted columns that match the expected names used during the training phase of the machine learning models such as "Heart Rate", "Temperature", "Oxygen Saturation", "Timestamp", and others. Any mismatch in column names or structure may cause the system to fail in reading or processing the data.
- **Availability of Pre-Trained Models:** The system assumes that all required machine learning models (e.g., Random Forest and XGBoost) have already been trained and are saved within the project directory in .pkl or .json formats. The application does not perform model training during runtime; it only loads and applies these pre-trained models.
- **System Runs Locally (Localhost):** The application is designed to run locally on the user's machine via localhost. It is not configured for deployment on external servers or cloud-based platforms. All data processing and computations are performed on the same device.
- **User Database Exists and Has Proper Structure:** It is assumed that the clinic_users.db database exists and includes a users table with fields like email, password, and role. If the database does not exist, the system will create it; however, the internal structure must remain consistent.
- **Presence of the uploads/ Folder:** The system assumes the existence of an uploads/ directory where CSV files uploaded by the user will be stored. If the folder is missing, the system will automatically create it, but its presence is crucial for data handling.
- **User Has Basic Understanding of System Usage:** It is assumed that the end user has a basic understanding of how to use the system specifically how to upload files, initiate analysis, and interpret basic results. The application does not include advanced tutorials or interactive guides, relying instead on its simple and intuitive interface.

- **The Host Device is Capable of Handling Processing Tasks:** It is assumed that the device running the system has sufficient computational resources (CPU and RAM) to process patient data and run machine learning models efficiently.

4.5 Code 1 Section A, B,C on APPENDIX

The code begins by importing a wide range of essential libraries that support building an integrated healthcare system leveraging artificial intelligence and data analysis. Core backend functionalities such as session management, request handling, and secure file uploads are facilitated by Flask, Flask-CORS, and Werkzeug, while the os library manages files and directories. SQLite3 handles local database interactions, and datetime formats timestamps for health records. For data processing and visualization, pandas, numpy, seaborn, and matplotlib are employed to analyze patients' vital signs effectively. Advanced AI capabilities are enabled through joblib and pickle for loading pre-trained machine learning models, alongside xgboost for building powerful predictive models assessing patient health status. The system architecture relies on two primary databases: clinic_users.db, which stores user login details including email, password, role, and PatientID linking patients to their health data; and patient_entry.db, which records comprehensive patient vital signs, medical history, prescribed medication, and calculated health indicators. The code dynamically manages database schemas by creating necessary tables, inserting default users, and updating tables to accommodate new fields without rebuilding the database, ensuring scalability and flexibility. The Flask application defines key routes, starting with a root / route redirecting users to the login page /login, where user credentials are verified against the database. Upon successful authentication, the system stores session data, including PatientID for patients, and redirects users to the home page; failed attempts result in error notifications. Together, these components form a cohesive, secure, and user-friendly platform enabling doctors and patients to interact smoothly with the healthcare system.

4.6 Code 1 Section H on APPENDIX

This section of the code manages the user registration process via the /create_account route. When accessed with a GET request, it displays the registration form; with a POST request, it processes the submitted data. The application validates the input fields—ensuring they are filled and that the password matches its confirmation—before attempting to save the new user's information in the clinic_users.db database. It handles common issues such as duplicate emails or server errors gracefully, providing clear feedback. Upon successful registration, the user is

redirected to the login page with a confirmation message, ensuring a smooth and secure sign-up experience.

4.7 Code 1 Section N on APPENDIX

The `/search_patient` route allows authenticated doctors and nurses to search for a patient's data using their PatientID. Upon a POST request, it retrieves the ID, queries the database for matching records, and returns a JSON response. If records are found, it extracts the latest vital info and calculated health metrics, compiles all unique medicines and medical histories, and provides a comprehensive view of the patient's data. If no data is found, it responds accordingly. This route ensures efficient access to up-to-date patient information.

4.8 Code 1 Section L ,M on APPENDIX

The `/save_patient` and `/upload` routes work together to enable secure and efficient entry of patient health data into the system. The `/save_patient` route handles manual form submissions via POST requests and is restricted to authorized users (doctors and nurses). It collects the patient's vital signs from the form, assigns a timestamp if none is provided, and inserts the data into the SQLite database. If successful, it returns a confirmation; otherwise, it handles errors gracefully. On the other hand, the `/upload` route manages the bulk import of patient data through CSV files. It validates the file format, saves it to the server, and uses Pandas to read its contents. The system checks for the presence of all required columns—including PatientID, HeartRate, BodyTemperature, and calculated fields like Derived_BMI and Derived_MAP—before inserting the data row by row into the database. Any formatting issues or missing fields trigger an error message. Together, these routes ensure both manual and bulk data entry are performed safely, accurately, and with proper access control.

4.9 Code 1 Section O on APPENDIX

The `/get_patient_data` route processes POST requests to retrieve patient health metrics for analysis or visualization, with access limited to doctors, nurses, and patients. Protected by the `@login_required` decorator, it enforces role-based access control. Patients can only access their own data using the PatientID stored in their session, while doctors and nurses must provide both a valid PatientID and a list of metrics in the request. The server validates the requested metrics and queries the `patient_entry.db` database for the corresponding data, returning timestamps and values in the response. Errors such as unauthorized access or missing data are handled with clear messages. This route ensures secure and structured access to patient records for clinical or analytical purposes.

Chapter 5: Results and Discussion

5.1 Presenting and Discussing the Results

During practical operation, the MedAnalyzer system demonstrated its ability to efficiently handle vital sign data entered either manually through forms or by uploading CSV files. Upon data entry, the system automatically calculated derived values such as Pulse Pressure, Body Mass Index (BMI), Mean Arterial Pressure (MAP), and Heart Rate Variability (HRV), offering a comprehensive overview of the patient's condition. Using the pre-trained XGBoost model, the system responded quickly by accurately predicting the patient's health risk level (High or Low) based on the entered vital signs. The Analytics page also played a key role in visualizing the progression of the patient's vital signs over time using interactive time-based charts, enabling doctors to monitor changes clearly. The results showed that the system effectively supports clinical decisions by providing accurate, interpretable predictions, and by generating detailed PDF medical reports that include analysis and medical recommendations—enhancing the efficiency of medical follow-ups within the system.

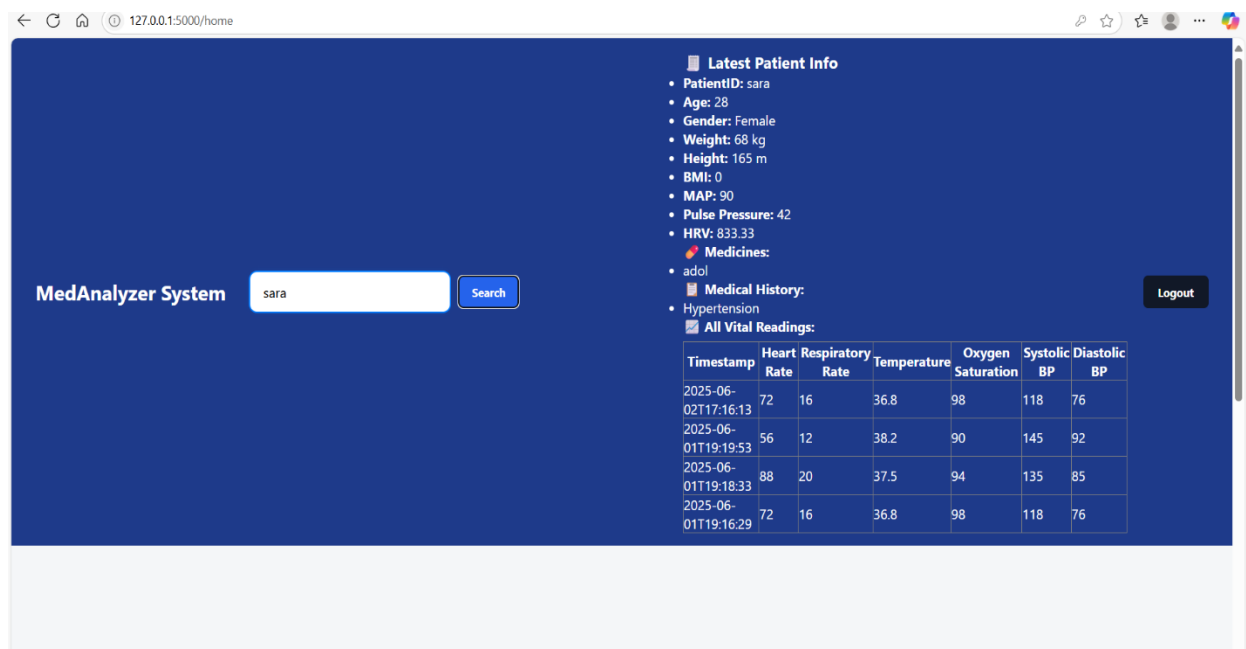


Figure 15: Patient search results by Patient Id.

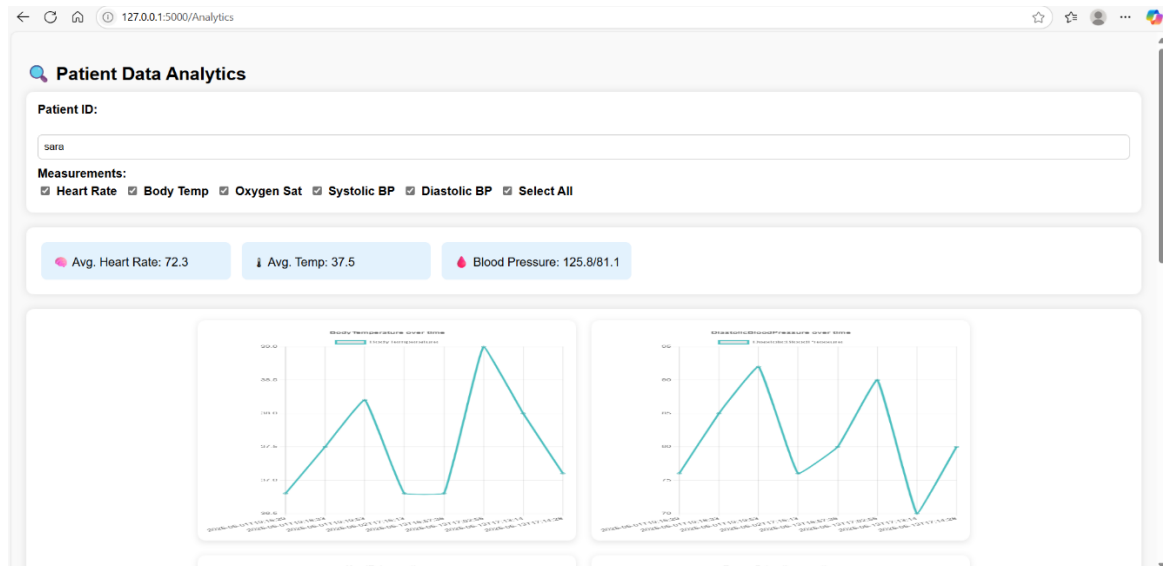


Figure16: Patient data result on plot at different times [1]



Figure 17: Patient data result on plot at different times [2]

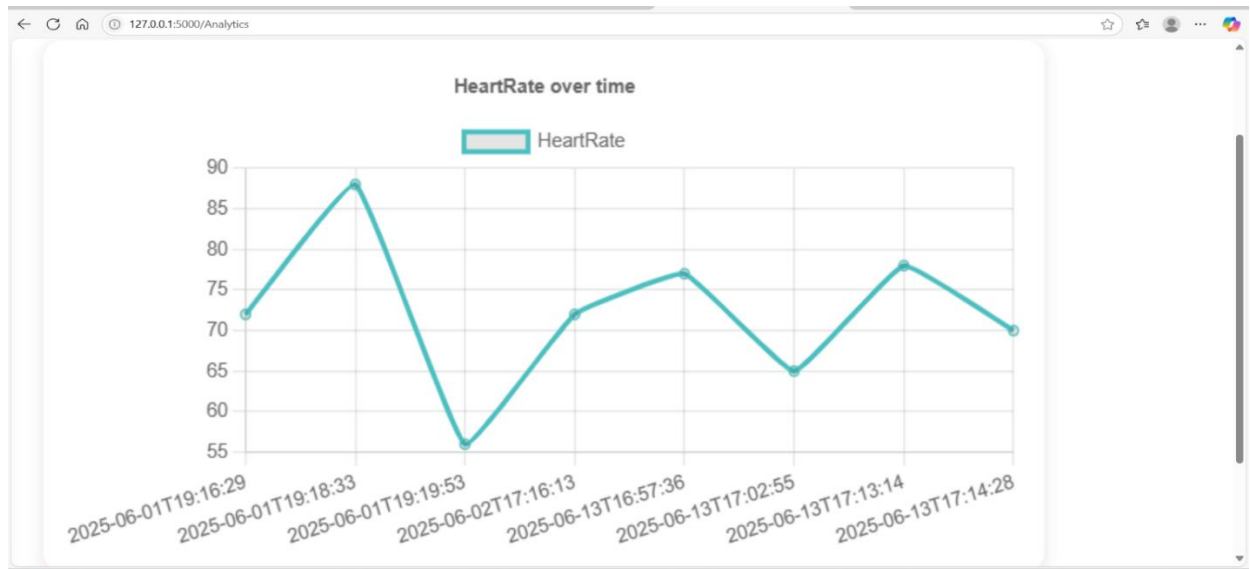


Figure 18: Patient data result on plot at different times [3]

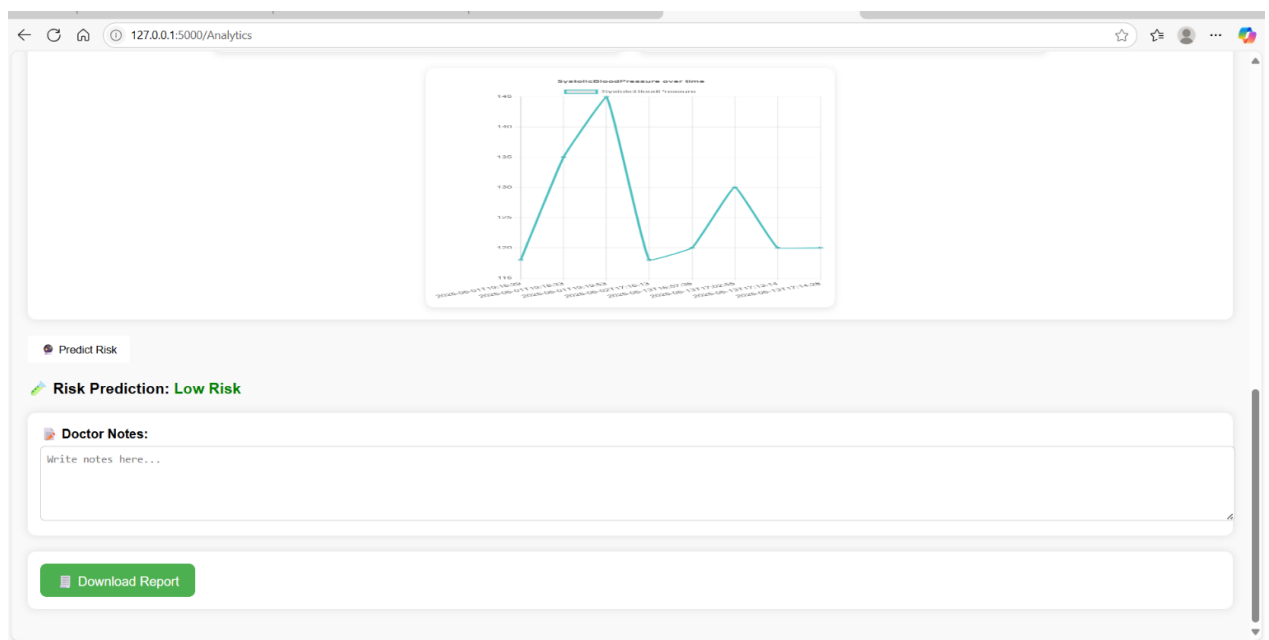


Figure 19: Prediction Results.

```

Confusion Matrix:
[[6153  11]
 [   6 6845]]
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [15:09:39] WARNING: /workspace/src/learner.cc:748:
Parameters: { "use_label_encoder" } are not used.

warnings.warn(msg, UserWarning)

Fold 5 Accuracy: 0.9992
Classification Report:
      precision    recall  f1-score   support

     0.0         1.00      1.00      1.00       6164
     1.0         1.00      1.00      1.00       6851

 accuracy         1.00      1.00      1.00      13015
  macro avg         1.00      1.00      1.00      13015
 weighted avg         1.00      1.00      1.00      13015

Confusion Matrix:
[[6162   2]
 [   9 6842]]

Mean CV Accuracy: 0.9982

```

Figure 20: Accuracy Result of predication Model.

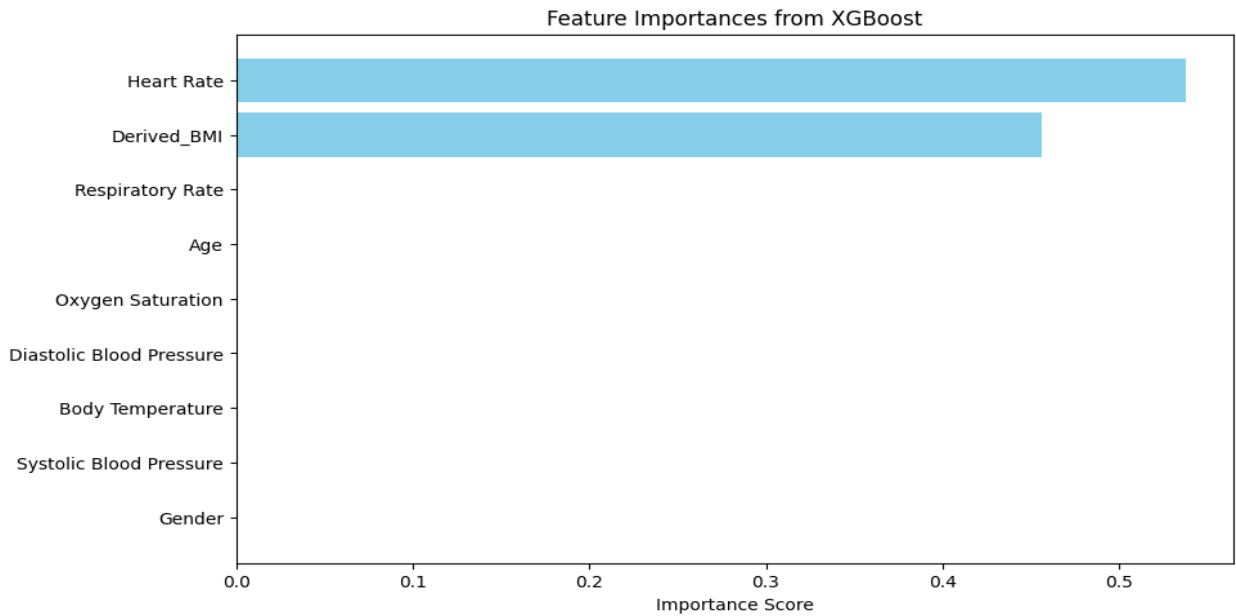


Figure 21: The feature importance in predication Model.

5.2 Strengths and Weaknesses of the System

5.2.1 Strengths

- **User-Friendly Interface:** The system was designed with simplicity and clarity in mind. The frontend, built using HTML, CSS, and JavaScript, allows users especially doctors to easily navigate the platform. Uploading patient data files, viewing graphs, and reading prediction results can be done with just a few clicks. This intuitive interaction eliminates the need for technical expertise, making the system accessible even to users with minimal technical background.
- **Fast and Efficient Processing:** Because the machine learning models used in the system (Random Forest and XGBoost) are already pre-trained and stored locally, the time between uploading a file and receiving a result is very short—typically within a few seconds. This speed is crucial in medical environments where timely decision-making can directly impact patient outcomes. The use of efficient Python libraries like Pandas, NumPy, and joblib also contributes to the rapid execution of data analysis and prediction.
- **High Prediction Accuracy:** The system leverages robust machine learning algorithms specifically XGBoost and Random Forest—that have demonstrated high performance in classifying health risks based on vital signs. These models were trained on medically relevant features, and proper preprocessing techniques were applied, including handling of missing data

and normalization. As a result, the predictions are often reliable in identifying whether a patient's condition is normal, moderate, or critical, giving doctors actionable insights.

- **Full Backend-Frontend Integration:** The backend, implemented with Flask and integrated with the frontend, works seamlessly to handle requests, process patient data, apply machine learning models, and return results to the interface. This tight integration ensures a smooth user experience without delays or errors. For example, once a CSV file is uploaded, Flask automatically processes the data and updates the dashboard with visual graphs and risk predictions.
- **Local Execution without the Need for Internet Access:** The system is designed to operate entirely on the local machine without requiring an internet connection during actual usage. All data analysis and prediction processes are performed within a local Python environment using pre-trained machine learning models. Databases and essential files are stored within the project directory, ensuring complete offline functionality. While an internet connection might be necessary during the initial setup phase—for example, to install required packages or libraries—the system itself does not rely on any external services or APIs during runtime. This enhances both data privacy and processing speed, which are critical in medical applications.

5.2.2 Weaknesses

- **Lack of Input Validation and Error Handling:** The system currently assumes that users will upload CSV files in the correct format with the necessary columns such as Heart Rate, Temperature, Timestamp, etc. However, if the uploaded file has missing or incorrectly named columns, the system may crash or behave unexpectedly. There is no comprehensive mechanism in place to check file integrity or alert the user with helpful error messages.
- **No Multi-user or Concurrent Access Support:** The application is designed as a single-user system running locally. It does not support simultaneous use by multiple doctors or users. There is no session management for different users or roles beyond the login database, and the system does not allow for collaborative analysis or shared records, which may be a limitation in larger clinical settings.
- **No Built-in Model Retraining Feature:** The current implementation relies on pre-trained machine learning models saved as .pkl or .json files. If there is a need to update the model using new patient data or improve performance through retraining, it must be done manually in a separate environment (e.g., Jupyter Notebook or external Python script). The system does not offer a user-facing interface or automated pipeline for updating the model with new data.

- **No Historical Data Storage or Reporting:** While the system provides real-time analysis for each uploaded file, it does not store previous analysis results or associate data with specific patients over time. There is no history or log of past sessions, and no reporting functionality to retrieve previous evaluations. This limits its ability to track patient progress or generate long-term insights unless results are saved manually by the user.

Chapter 6: Economical, Ethic, and Contemporary Issues

6.1 Preliminary Cost Estimation and Justification

The MedAnalyzer System is a web-based medical data analysis platform that offers functionalities such as patient data input, vital signs tracking, risk prediction, and report generation. The preliminary cost estimate includes:

- **Development Tools:** Open-source tools like Flask (backend), SQLite (database), Chart.js (visualization), and HTML/CSS/JavaScript (frontend) were used, minimizing licensing costs.
- **Labor:** Assuming a team of 2–3 developers working over 3–4 months, the estimated development cost would range between **\$5,000 to \$10,000**, depending on hourly rates.
- **Hosting and Deployment:** Cloud services such as Heroku, AWS, or a VPS would cost approximately **\$20–\$50/month**.
- **Maintenance and Updates:** Estimated at **10–15%** of the initial cost per year.

The cost is justified due to the system’s potential to streamline medical data analysis, assist in risk management, and improve healthcare decision-making, particularly in under-resourced hospitals and clinics.[13][14][15]

6.2 Relevant Codes of Ethics and Moral Frameworks

The development and deployment of MedAnalyzer System adhere to internationally recognized ethical principles, including:

- **ACM Code of Ethics:** Ensuring privacy, data integrity, and professional responsibility.
- **HIPAA (for reference):** Though not binding in Jordan, the system considers HIPAA-like principles such as confidentiality and secure data handling.
- **Jordan Medical Association’s Ethical Guidelines:** Respect for patient data, professionalism, and informed use of technology in healthcare.

The system is designed with role-based access control, data encryption, and audit logging to ensure adherence to ethical practices in patient data management.[13][14][15]

6.3 Ethical Dilemmas and Justification of Proposed Solution

A major ethical dilemma is the **handling of sensitive patient data**. Improper access or data breaches could lead to harm or misuse.

Solution:

- Only authorized users (doctors, admin) can access sensitive data.
- Access is verified through Flask-based role management.
- Sensitive fields are encrypted and stored securely.
- Patient data used for analysis or model training is anonymized to preserve privacy.

Another dilemma involves **AI-based risk predictions**, which could influence critical decisions.

Solution: The model is presented as a **supporting tool**, not a final decision-maker, and always leaves the final call to the healthcare provider. [13][14][15]

6.4 Relevant Environmental Considerations

Although MedAnalyzer is primarily a digital product, some environmental aspects are considered:

- **Digital over Paper:** Encourages digital reports and data storage, reducing paper consumption.
- **Cloud Hosting Efficiency:** Using green-certified data centers (if possible) can minimize the carbon footprint.
- **Energy Efficiency:** The system is lightweight and optimized for performance, reducing unnecessary resource usage on client and server machines.

6.5 Relevance to Jordan and Region (Social, Cultural, and Political)

- **Social Relevance:** MedAnalyzer supports medical staff in public and private healthcare sectors, especially where resources are limited. It empowers hospitals to make data-driven decisions efficiently.
- **Cultural Relevance:** The system can be localized into Arabic, respecting local language needs. It can also be customized to align with local medical practices and reporting standards.
- **Political Relevance:** In line with Jordan's digital health transformation initiatives and national e-health strategies, MedAnalyzer complements government efforts to modernize healthcare systems. [13][14][15]

6.6 Other Issues and Constraints

- **Legal Constraints:** While Jordan lacks strict data protection laws like GDPR, the system is built with international standards in mind to anticipate future regulations.
- **Scalability:** Initially built for small to medium-scale hospitals or clinics, future upgrades may be needed for larger institutions.
- **User Training:** Medical staff may require basic digital literacy training to use the system effectively.

- **Internet Dependence:** The system's performance relies on stable internet connections, which may not always be available in remote areas.

Chapter 7: Project Management

7.1 Schedule and Time Management of the Project

In the **MedAnalyzer System** project, a clear timeline was developed for each stage to ensure deadline adherence and goal achievement within the allocated period. The project was divided into major phases as follows:

- **Requirement Analysis (Week 1–2):** Gathering and analyzing system needs from end users and academic supervisor.
- **UI Design (Week 3–5):** Designing pages using HTML/CSS with wireframes and user interface prototypes.
- **Back-End Development using Flask (Week 6-9):** Creating REST APIs, integrating SQLite database, and enabling input, display, and analysis features.
- **Data Analysis Module Development (Week 10):** Training the predictive model and integrating risk level prediction along with calculated vital indicators such as HRV and MAP.
- **Testing and Debugging (Week 11):** Testing the system using various scenarios and fixing identified bugs.
- **Final Report and Presentation (Week 12-13):** Writing the final report and preparing the presentation.

Tasks were scheduled using a Gantt chart, and tools like Trello and Google Calendar were used for time management and progress tracking.

7.2 Resource and Cost Management

Human Resources:

- **Front-End Developer:** Responsible for UI design using HTML, CSS, and JavaScript.
- **Back-End Developer:** Developed the server side using Python and Flask, and integrated database and model prediction.
- **Data Analyst:** Developed the risk prediction model and implemented vital sign calculations.
- **Project Supervisor:** Reviewed deliverables and guided the team.

Technical Resources:

- Personal computers for development and testing.
- Free development tools, such as:
 - VS Code, Git, Flask, SQLite, Chart.js, jsPDF.
- Google Drive and GitHub for file storage and collaboration.

Estimated Cost:

Since the project relied on free/open-source tools and student effort, the total cost was **very minimal**:

- Internet subscription: \$10/month.
- Optional resources (icons, templates): Around \$15 (optional).

7.3 Quality Management

Several measures were taken to ensure system quality:

- **Functional testing for each page:** Verifying that all features (input, analysis, search, registration, etc.) worked properly.
- **Invalid input testing:** Checking how the system handles incorrect or missing data.
- **Code documentation:** Organized and commented code for easy maintenance.
- **User Experience (UX) improvements:** Interfaces were designed to be clear and user-friendly.
- **Code review:** Code was reviewed by team members and the academic supervisor.

7.4 Risk Management

The main project risks were identified, and mitigation plans were developed:

Risk	Impact Level	Mitigation Strategy
Lack of dataset quality	High	Use verified medical datasets, clean data before use, and validate inputs.
Task delay	Medium	Weekly review of schedule and fair task distribution.
Code malfunction or bugs	High	Unit testing each component and clear installation instructions.
Poor team communication	Low	Frequent communication via WhatsApp .

Table 4: risk management plan

7.5 Project Procurement

The project did not require major purchases, but some procurement considerations were necessary:

- **Software and tools:** All used tools were free or open source.
- **Educational resources:** Some academic articles and references were downloaded to study predictive models and vital signs.
- **Hardware:** Team members used their personal computers. No extra devices were required.

Chapter 8: Conclusion and Future Work

8.1 Summary of the Main Contributions

The **MedAnalyzer** system was developed as a comprehensive platform for medical data analysis. It enables healthcare professionals to input, process, and visualize vital signs either manually or via CSV upload. The system automatically calculates key health metrics such as Body Mass Index (BMI), Mean Arterial Pressure (MAP), and Heart Rate Variability (HRV). It includes a graphical dashboard for visualization, supports PDF report generation, and incorporates a machine learning model for health risk prediction. Furthermore, user authentication is implemented to ensure secure access, making the system efficient and user-friendly in medical settings.

8.2 Future Work

To extend the functionality and impact of the MedAnalyzer system, the following improvements are proposed:

- **Wearable Device Integration:** Real-time data collection through smartwatches and medical wearables for continuous patient monitoring.
- **Mobile Application:** Development of a mobile version to facilitate remote access for both doctors and patients.
- **AI-Based Diagnostic Assistance:** Enhancement of the current model to provide preliminary diagnostic suggestions using advanced machine learning techniques.
- **Multilingual Interface:** Support for multiple languages to increase accessibility in diverse regions.
- **Cloud Infrastructure:** Migration to cloud platforms (e.g., AWS, Google Cloud) to enable scalability and global access.
- **Electronic Health Record (EHR) Integration:** Synchronization with hospital EHR systems for seamless data exchange.
- **Compliance with Data Protection Standards:** Implementation of GDPR and HIPAA protocols for legal and ethical data handling.
- **Automated Alerts:** Notification system for abnormal readings to promptly alert medical personnel.

9.3 Lessons Learned

The development of the MedAnalyzer system yielded several key lessons:

- Effective **user interface design** is crucial to ensure accessibility for non-technical medical personnel.
- Proper **integration between backend and frontend** technologies (e.g., Flask with HTML/JavaScript) is essential for interactive functionality.
- Applying **machine learning in healthcare** necessitates careful data preprocessing and ethical considerations to avoid bias and ensure accurate predictions.
- Strong **project management skills**, including time coordination and task delegation, are vital for successful project delivery.

References

- [1] Flask, *Flask Documentation* [Online]. Available: <https://flask.palletsprojects.com>
- [2] Python Software Foundation, *Python Language Reference*, ver. 3.x. [Online]. Available: <https://www.python.org/doc/>
- [3] W. McKinney, *Python for Data Analysis*. Sebastopol, CA: O'Reilly Media, 2012.
- [4] NumPy Developers, *NumPy Documentation* [Online]. Available: <https://numpy.org/>
- [5] pandas developers, *pandas Documentation* [Online]. Available: <https://pandas.pydata.org/>
- [6] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [7] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [8] D. R. Hipp, *SQLite: Embedded Database Engine* [Online]. Available: <https://sqlite.org/>
- [9] Chart.js Team, *Chart.js Documentation* [Online]. Available: <https://www.chartjs.org/>
- [10] Parallax, *jsPDF: Generate PDF files in JavaScript* [Online]. Available: <https://github.com/parallax/jsPDF>
- [11] Microsoft, *Visual Studio Code* [Online]. Available: <https://code.visualstudio.com/>
- [12] Mozilla Developer Network (MDN), *HTML, CSS, and JavaScript Documentation* [Online]. Available: <https://developer.mozilla.org/>
- [13] Jordan Medical Association, *Code of Conduct and Medical Ethics*, 2020. [Online]. Available: <http://www.jma.org.jo>
- [14] Ministry of Health – Jordan, *National Strategy for Digital Transformation in Health*, 2023. [Online]. Available: <https://www.moh.gov.jo>
- [15] Association for Computing Machinery, *ACM Code of Ethics and Professional Conduct*, 2018. [Online]. Available: <https://www.acm.org/code-of-ethics>
- [16] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.
- [17] A. Esteva et al., “A guide to deep learning in healthcare,” *Nature Med.*, vol. 25, no. 1, pp. 24–29, Jan. 2019, doi: 10.1038/s41591-018-0316-0.
- [18] U.S. Department of Health & Human Services, “Health Insurance Portability and Accountability Act (HIPAA),” 1996. [Online]. Available: <https://www.hhs.gov/hipaa>

APPENDIX A: User Manual

Code 1 python (flask)→Backend

Section A

```
from flask import Flask, render_template, request, redirect, url_for, flash, session, jsonify
from flask_cors import CORS
from werkzeug.utils import secure_filename
from datetime import datetime
from functools import wraps
import os
import sqlite3
import pandas as pd
import numpy as np
import joblib
import matplotlib.pyplot as plt
import seaborn as sns
from flask import send_file
import io
import base64
import pickle
import matplotlib.pyplot as plt
import matplotlib
matplotlib.use('Agg')
import pickle
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.preprocessing import LabelEncoder
import pickle
import json
from functools import wraps
import xgboost as xgb

app = Flask(__name__)
app.secret_key = 'secret-key'

UPLOAD_FOLDER = 'uploads'
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

CORS(app)
```

Section B

```
NEW_DB_PATH1 = 'clinic_users.db'

def init_new_db():
    with sqlite3.connect(NEW_DB_PATH1) as conn:
        cursor = conn.cursor()

        cursor.execute("""
            CREATE TABLE IF NOT EXISTS users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                email TEXT UNIQUE NOT NULL,
                password TEXT NOT NULL,
                role TEXT NOT NULL
            )
```

```
)
"""
print("✅ تم إنشاء قاعدة البيانات الجديدة وجدول users.")
```

```
users = [
    ('mohamed.ahmed@clinic.com', '123456', 'doctor'),
    ('sarah.ali@clinic.com', '123456', 'doctor'),
    ('khaled.mansour@clinic.com', '123456', 'doctor'),
    ('asmaa.ali@clinic.com', '123456', 'doctor'),
    ('sora.ali@clinic.com', '123456', 'doctor'),
    ('tamara.mwafaq@clinic.com', '123456', 'doctor'),
    ('amal.ayman@clinic.com', '123456', 'doctor'),
    ('deema.alkhalidi@clinic.com', '123456', 'doctor'),
    ('ali.ahmad@clinic.com', '123456', 'doctor'),
    ('alaa.youssef@clinic.com', '123456', 'doctor'),
    ('youssef.ameen@clinic.com', '123456', 'doctor'),
    ('khalid.ahmad@clinic.com', '123456', 'doctor'),
    ('olah.ahmad@clinic.com', '123456', 'doctor'),
    ('sawsan.ameer@clinic.com', '123456', 'doctor'),
    ('ameer.fouad@clinic.com', '123456', 'doctor'),
    ('areen.ali@clinic.com', '123456', 'doctor'),
    ('mohammad.ibraheem@clinic.com', '123456', 'doctor'),
    ('mahmood.esaam@clinic.com', '123456', 'doctor'),
    ('rami.mohammad@clinic.com', '123456', 'doctor'),
    ('mohammad.khaled@clinic.com', '123456', 'doctor')
]
```

```
for email, password, role in users:
    cursor.execute("SELECT * FROM users WHERE email = ?", (email,))
    if not cursor.fetchone():
        cursor.execute("INSERT INTO users (email, password, role) VALUES (?, ?, ?)", (email, password, role))
```

```
print("✅ تم التأكد من إضافة جميع المستخدمين الافتراضيين.")
```

```
init_new_db()
```

```
NEW_DB_PATH1 = 'clinic_users.db'
```

```
def add_patient_id_column():
    with sqlite3.connect(NEW_DB_PATH1) as conn:
        cursor = conn.cursor()
```

```
cursor.execute("PRAGMA table_info(users)")
columns = [col[1] for col in cursor.fetchall()]
```

```
if 'PatientID' not in columns:
    cursor.execute("ALTER TABLE users ADD COLUMN PatientID TEXT")
    print("✅ تم إضافة عمود patient_id إلى جدول users.")
else:
    print("✅ users موجود مسبقاً في جدول patient_id عمود")
```

```
def init_new_db_with_patient_id():
    with sqlite3.connect(NEW_DB_PATH1) as conn:
        cursor = conn.cursor()
```

```

cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        email TEXT UNIQUE NOT NULL,
        password TEXT NOT NULL,
        role TEXT NOT NULL
    )
""")
print("✅ (إن لم يكن موجوداً) users تم إنشاء جدول")

add_patient_id_column()

# patient_id أو " إن لم يكن للمستخدم None ضع patient_id المستخدمين الافتراضيين مع
users = [
    ('mohamed.ahmed@clinic.com', '123456', 'doctor', None),
    ('sarah.ali@clinic.com', '123456', 'doctor', None),
    ('khaled.mansour@clinic.com', '123456', 'doctor', None),
    # مثال لمريض:
    ('patient1@example.com', 'pass123', 'patient', 'P001'),
    ('patient2@example.com', 'pass456', 'patient', 'P002'),
]
for email, password, role, PatientID in users:
    cursor.execute("SELECT * FROM users WHERE email = ?", (email,))
    if not cursor.fetchone():
        cursor.execute(
            "INSERT INTO users (email, password, role, PatientID) VALUES (?, ?, ?, ?)",
            (email, password, role, PatientID)
        )
    print(f"✅ {email} تمت إضافة المستخدم")

print("✅ PatientID. تم التأكد من إضافة جميع المستخدمين الافتراضيين مع")

```

init_new_db_with_patient_id()

NEW_DB_PATH = "patient_entry.db"

```

def init_new_patient_db():
    try:
        with sqlite3.connect(NEW_DB_PATH) as conn:
            cursor = conn.cursor()
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS patients (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    PatientID TEXT NOT NULL,
                    HeartRate REAL,
                    RespiratoryRate REAL,
                    Timestamp TEXT,
                    BodyTemperature REAL,
                    OxygenSaturation REAL,
                    SystolicBloodPressure REAL,
                    DiastolicBloodPressure REAL,
                    Age INTEGER,
                    Gender TEXT,
                    Weight_kg REAL,

```

```

        Height_m REAL,
        Derived_HRV REAL,
        Derived_Pulse_Pressure REAL,
        Derived_BMI REAL,
        Derived_MAP REAL
    )
    """
    conn.commit()
    print("✅ بنجاح 'patients' وجدول 'patient_entry.db' تم إنشاء قاعدة البيانات")
except sqlite3.Error as e:
    print("❌ حدث خطأ أثناء إنشاء قاعدة البيانات", e)

```

NEW_DB_PATH = "patient_entry.db"

```

def alter_patient_table():
    try:
        with sqlite3.connect(NEW_DB_PATH) as conn:
            cursor = conn.cursor()

            # إذا لم يكن موجودًا medicine أضف عمود
            cursor.execute("PRAGMA table_info(patients)")
            columns = [col[1] for col in cursor.fetchall()]

            if "medicine" not in columns:
                cursor.execute("ALTER TABLE patients ADD COLUMN medicine TEXT")
                print("✅ تم إضافة العمود 'medicine'.")

            if "patient_history" not in columns:
                cursor.execute("ALTER TABLE patients ADD COLUMN patient_history TEXT")
                print("✅ تم إضافة العمود 'patient_history'.")

            conn.commit()
            print("🔄 تم تعديل جدول 'patients' بنجاح.")
    except sqlite3.Error as e:
        print("❌ حدث خطأ أثناء تعديل الجدول", e)

```

alter_patient_table()

Section C

```

@app.route("/")
def index():
    return redirect(url_for("login"))

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        print("✅ Login Success | Role:", session.get('role'))
        print("✅ Session Data:", dict(session))

    if request.is_json:

```

```

data = request.get_json()
email = data.get('email')
password = data.get('password')
role = data.get('role')
patient_id = data.get('patient_id')

with sqlite3.connect(NEW_DB_PATH1) as conn:
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE email=? AND password=?", (email, password))
    user = cursor.fetchone()

    if user:

        if user[3] != role:
            return jsonify({'success': False, 'message': 'Invalid role for this account. ⚠'}, 401)

        session['email'] = email
        session['role'] = user[3]
        if user[3] == 'patient' and patient_id:
            session['PatientID'] = patient_id

        return jsonify({'success': True})
    else:
        return jsonify({'success': False, 'message': 'The Email Or Password IS Incorrect ✖'}, 401)
else:
    email = request.form['email']
    password = request.form['password']
    role = request.form.get('role')
    patient_id = request.form.get('patient_id')

    with sqlite3.connect(NEW_DB_PATH1) as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM users WHERE email=? AND password=?", (email, password))
        user = cursor.fetchone()

        if user:
            session['email'] = email
            session['role'] = user[3]
            if user[3] == 'patient' and patient_id:
                session['PatientID'] = user[4]
            flash("Login successful.")
            return redirect(url_for('home'))
        else:
            flash("Invalid email or password")
            return render_template("login.html")

return render_template("login.html")

```

Section D

```

@app.route('/forgot_password', methods=['GET', 'POST'])
def forgot_password():
    if request.method == 'POST':
        data = request.get_json()

```

```

email = data.get('email')
new_password = data.get('new_password')
with sqlite3.connect(NEW_DB_PATH1) as conn:
    cursor = conn.cursor()
    cursor.execute('UPDATE users SET password=? WHERE email=?', (new_password, email))
    if cursor.rowcount == 0:
        return jsonify({'success': False, 'message': 'Email not found.'})
    else:
        conn.commit()
        return jsonify({'success': True, 'message': 'Password changed successfully .'})
return render_template('forgot_password.html')

```

Section E

```

@app.route('/home')
def home():
    if "email" not in session:
        flash(" You must log in first .", "warning")
        return redirect(url_for("login"))
    return render_template("Dashboard.html", role=session.get("role"))

```

Section F

```

def login_required(roles=None):
    def wrapper(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if 'role' not in session:

                return jsonify({'error': 'Unauthorized. Please log in.'}), 401

            if roles and session['role'] not in roles:

                return jsonify({'error': 'Access denied.'}), 403

            return f(*args, **kwargs)
        return decorated_function
    return wrapper

```

Section G

```

@app.route('/Doctors_page')
@login_required(roles='doctor')
def Doctors_page():
    return render_template('Doctors_page.html')

```

Section H

```

@app.route('/create_account', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':

```

```

email = request.form.get('email')
password = request.form.get('password')
confirm_password = request.form.get('confirm_password')
role = request.form.get('role')

print(f"Trying to create account with: {email}, {role}")

if not email or not password or not confirm_password or not role:
    flash("All fields are required.", "danger")
    return redirect(url_for("register"))

if password != confirm_password:
    flash("Passwords do not match.", "danger")
    return redirect(url_for("register"))

with sqlite3.connect(NEW_DB_PATH1) as conn:
    cursor = conn.cursor()
    try:
        cursor.execute(
            'INSERT INTO users (email, password, role) VALUES (?, ?, ?)',
            (email, password, role)
        )
        conn.commit()
        flash('Account created successfully.', 'success')
        return redirect(url_for('login'))

    except sqlite3.IntegrityError:
        flash('Email is already registered.', 'danger')
    except Exception as e:
        print("Unexpected error:", e)
        flash(f"Server error: {str(e)}", 'danger')

return render_template('create_account.html')

```

Section K

```

@app.route("/about")
def about():
    user_role = session.get('role')
    if user_role not in ['doctor', 'Nurse']:
        flash("Access denied: only doctors and nurses can view this page.", "danger")
        return redirect(url_for('login'))

    return render_template("about.html")

```

```
NEW_DB_PATH = "patient_entry.db"
```

```

def allowed_roles():
    user_role = session.get('role')
    return user_role in ['doctor', 'Nurse']

```

```

@app.route("/patient_data_entry", methods=["GET"])
def patient_data_entry():
    if not allowed_roles():
        flash("Access denied: only doctors and nurses are allowed.", "danger")

```

```

    return redirect(url_for('login'))
return render_template("patient_data_entry.html")

```

Section L

```

@app.route("/save_patient", methods=["POST"])
def save_patient():
    if not allowed_roles():
        flash("Access denied: only doctors and nurses are allowed.", "danger")
        return redirect(url_for('login'))

    try:
        data = request.form
        timestamp = data.get('Timestamp') or datetime.now().isoformat()

        with sqlite3.connect(NEW_DB_PATH) as conn:
            cursor = conn.cursor()
            cursor.execute("""
                INSERT INTO patients (
                    PatientID, HeartRate, RespiratoryRate, Timestamp, BodyTemperature,
                    OxygenSaturation, SystolicBloodPressure, DiastolicBloodPressure,
                    Age, Gender, Weight_kg, Height_m, Derived_HRV,
                    Derived_Pulse_Pressure, Derived_BMI, Derived_MAP,
                    medicine, patient_history
                ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            """, (
                data.get('PatientID'),
                data.get('HeartRate'),
                data.get('RespiratoryRate'),
                timestamp,
                data.get('BodyTemperature'),
                data.get('OxygenSaturation'),
                data.get('SystolicBloodPressure'),
                data.get('DiastolicBloodPressure'),
                data.get('Age'),
                data.get('Gender'),
                data.get('Weight_kg'),
                data.get('Height_m'),
                data.get('Derived_HRV'),
                data.get('Derived_Pulse_Pressure'),
                data.get('Derived_BMI'),
                data.get('Derived_MAP'),
                data.get('medicine'),
                data.get('patient_history')
            ))
            conn.commit()
        return 'Saved.', 200
    except Exception as e:
        print("An error occurred while saving the data. ✕ :", e)
        return 'Server error', 500

```

Section M

```

@app.route('/upload', methods=['POST'])
def upload_file():
    if 'file-upload' not in request.files:
        flash('No file part')
        return redirect(request.url)

    file = request.files['file-upload']

    if file.filename == "":
        flash('No selected file')
        return redirect(request.url)

    if file and file.filename.endswith('.csv'):
        filename = secure_filename(file.filename)
        filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
        file.save(filepath)

        try:
            df = pd.read_csv(filepath)

            required_columns = [
                'PatientID', 'HeartRate', 'RespiratoryRate', 'Timestamp', 'BodyTemperature',
                'OxygenSaturation', 'SystolicBloodPressure', 'DiastolicBloodPressure',
                'Age', 'Gender', 'Weight_kg', 'Height_m', 'Derived_HRV',
                'Derived_Pulse_Pressure', 'Derived_BMI', 'Derived_MAP'
            ]

            if not all(col in df.columns for col in required_columns):
                flash('Missing required columns in the uploaded file.')
                return redirect(request.url)

            with sqlite3.connect('patient_entry.db') as conn:
                cursor = conn.cursor()
                for _, row in df.iterrows():
                    timestamp = row['Timestamp']
                    cursor.execute("""
                        INSERT INTO patients (
                            PatientID, HeartRate, RespiratoryRate, Timestamp, BodyTemperature,
                            OxygenSaturation, SystolicBloodPressure, DiastolicBloodPressure,
                            Age, Gender, Weight_kg, Height_m, Derived_HRV,
                            Derived_Pulse_Pressure, Derived_BMI, Derived_MAP
                        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        row['PatientID'],
                        row['HeartRate'],
                        row['RespiratoryRate'],
                        timestamp,
                        row['BodyTemperature'],
                        row['OxygenSaturation'],
                        row['SystolicBloodPressure'],
                        row['DiastolicBloodPressure'],
                        row['Age'],
                        row['Gender'],
                        row['Weight_kg'],
                        row['Height_m'],

```

```

        row['Derived_HRV'],
        row['Derived_Pulse_Pressure'],
        row['Derived_BMI'],
        row['Derived_MAP']
    ))
    conn.commit()

    flash('File uploaded and data inserted successfully.')
    return redirect(url_for('index'))

except Exception as e:
    flash(f'Error processing file: {str(e)}')
    return redirect(request.url)

else:
    flash('Invalid file type. Please upload a CSV file.')
    return redirect(request.url)

```

Section N

```

@app.route("/search_patient", methods=["POST"])
@login_required(roles=['doctor', 'Nurse'])
def search_patient():
    PatientID = request.form.get("PatientID")
    print("Received PatientID:", PatientID)

    if not PatientID:
        print("No Patient ID provided")
        return jsonify({"found": False})

    conn = sqlite3.connect(NEW_DB_PATH)
    conn.row_factory = sqlite3.Row
    c = conn.cursor()

    c.execute("SELECT * FROM patients WHERE PatientID = ? ORDER BY Timestamp DESC", (PatientID,))
    rows = c.fetchall()

    conn.close()

    if not rows:
        print("No records found for patient")
        return jsonify({"found": False})

    latest = dict(rows[0])
    all_readings = [dict(row) for row in rows]

    all_medicines = set()
    all_histories = set()

    for row in all_readings:
        if row.get("medicine"):
            meds = [m.strip() for m in row["medicine"].split(",")]
            all_medicines.update(meds)
        if row.get("patient_history"):
            histories = [h.strip() for h in row["patient_history"].split(",")]
            all_histories.update(histories)

    return jsonify({
        "found": True,
        "latest_info": {

```

```

        "PatientID": latest["PatientID"],
        "Age": latest["Age"],
        "Gender": latest["Gender"],
        "Weight_kg": latest["Weight_kg"],
        "Height_m": latest["Height_m"],
        "Derived_BMI": latest["Derived_BMI"],
        "Derived_MAP": latest["Derived_MAP"],
        "Derived_Pulse_Pressure": latest["Derived_Pulse_Pressure"],
        "Derived_HRV": latest["Derived_HRV"]
    },
    "all_readings": all_readings,
    "all_medicines": sorted(all_medicines),
    "all_histories": sorted(all_histories)
})

```

Section O

```

@app.route('/get_patient_data', methods=['POST'])
@login_required(roles=['doctor', 'Nurse', 'patient']) # السماح للأدوار المطلوبة
def get_patient_data():
    user_role = session.get('role')
    print("DEBUG: session role =", session.get('role'))

    if user_role == 'patient':
        requested_patient_id = session.get('PatientID')
        data = request.get_json(silent=True) or {}
        if data and 'patient_id' in data and data['patient_id'] != requested_patient_id:
            return jsonify({'error': 'Access denied: Patients can only view their own data'}), 403
        metrics = [
            "HeartRate", "RespiratoryRate", "BodyTemperature",
            "OxygenSaturation", "SystolicBloodPressure", "DiastolicBloodPressure",
            "Age", "Gender", "Derived_HRV", "Derived_Pulse_Pressure",
            "Derived_BMI", "Derived_MAP"
        ]
        requested_patient_id = requested_patient_id

    elif user_role in ['doctor', 'Nurse']:
        data = request.get_json()
        requested_patient_id = data.get('patient_id')
        metrics = data.get('metrics')

        if not requested_patient_id or not metrics:
            return jsonify({'error': 'Patient ID and metrics are required'}), 400

    else:
        return jsonify({'error': 'Access denied'}), 403

    result = {metric: [] for metric in metrics}
    result['timestamps'] = []

    try:
        with sqlite3.connect('patient_entry.db') as conn:

```

```

cursor = conn.cursor()
safe_metrics = [m for m in metrics if isinstance(m, str) and m.isidentifier()]
if not safe_metrics:
    return jsonify({'error': 'Invalid metrics'}), 400

query = f"""
SELECT Timestamp, {' '.join(safe_metrics)}
FROM patients
WHERE PatientID = ?
ORDER BY Timestamp
"""

rows = cursor.execute(query, (requested_patient_id,)).fetchall()
if not rows:
    return jsonify({'error': "No data found for this patient"}), 404

for row in rows:
    result['timestamps'].append(row[0])
    for i, metric in enumerate(safe_metrics):
        val = row[i + 1]
        result[metric].append(val if val is not None else 0)

except Exception as e:
    return jsonify({'error': f"Internal Server Error: {str(e)}"}), 500

return jsonify(result)

```

Section P

```

model = xgb.XGBClassifier()
model.load_model('final_xgb_model (4).json')
scaler = joblib.load('scaler (4).pkl')

@app.route('/predict_risk', methods=['POST'])
@login_required()
def predict_risk():
    patient_id = None

    if session['role'] == 'patient':
        patient_id = session.get('PatientID')
    else:
        data = request.get_json(silent=True)
        patient_id = data.get('patient_id') if data else None

    if not patient_id:
        return jsonify({'error': 'Patient ID is required'}), 400

    try:
        with sqlite3.connect("patient_entry.db") as conn:
            cursor = conn.cursor()
            cursor.execute("""
                SELECT
                    HeartRate, RespiratoryRate, BodyTemperature,
                    OxygenSaturation, SystolicBloodPressure, DiastolicBloodPressure,
                    Age, Gender, Derived_BMI
                FROM patients
                WHERE PatientID = ?
                ORDER BY Timestamp DESC
                LIMIT 1
            """)

```

```

    "", (patient_id,))
    row = cursor.fetchone()

    if not row:
        return jsonify({'error': 'No data found for this patient'}), 404

    gender_value = row[7]
    gender_num = 1 if str(gender_value).lower() == 'male' else 0

    model_input = {
        "HeartRate": float(row[0]),
        "RespiratoryRate": float(row[1]),
        "BodyTemperature": float(row[2]),
        "OxygenSaturation": float(row[3]),
        "SystolicBloodPressure": float(row[4]),
        "DiastolicBloodPressure": float(row[5]),
        "Age": float(row[6]),
        "Gender": gender_num,
        "Derived_BMI": float(row[8])
    }

    input_df = pd.DataFrame([model_input])

    column_mapping = {
        "HeartRate": "Heart Rate",
        "RespiratoryRate": "Respiratory Rate",
        "BodyTemperature": "Body Temperature",
        "OxygenSaturation": "Oxygen Saturation",
        "SystolicBloodPressure": "Systolic Blood Pressure",
        "DiastolicBloodPressure": "Diastolic Blood Pressure",
        "Derived_BMI": "Derived_BMI",
        "Age": "Age",
        "Gender": "Gender"
    }
    input_df.rename(columns=column_mapping, inplace=True)

    input_scaled = scaler.transform(input_df)

    prediction = model.predict(input_scaled)[0]
    result = "High Risk" if prediction == 1 else "Low Risk"

    return jsonify({
        'risk_prediction': result
    })

except Exception as e:
    print(f"Error in prediction route: {e}")
    return jsonify({'error': f"Server error: {str(e)}"}), 500

```

Section Q

```

@app.route('/Analytics')
def Analytics():
    role = session.get('role')

    if not role:
        flash("Please login to access analytics.")
        return redirect(url_for('login'))

    patient_id = None
    print("✅ Analytics route accessed.")
    if role == 'patient':
        patient_id = session.get('PatientID')
        if not patient_id:
            flash("Missing patient ID.")
            return redirect(url_for('login'))

    return render_template('Analytics.html', patient_id=patient_id, role=role)

```

Section R

```

@app.route('/logout', methods=['POST'])
def logout():
    session.clear()
    return redirect('/')

if __name__ == "__main__":
    init_new_patient_db()
    app.run(debug=True)

```

codes (HTML,CSS,JAVASCRIPT)→Frontend

code 2→LOGIN page

```

<!DOCTYPE html>
<html lang="en">
<head>
    <title>Login-MedAnalyzer System</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Abril+Fatface&family=Caveat:wght@400..700&family=Oswald:wght@200..700&display=swap" rel="stylesheet">
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.0/css/all.min.css">
    <link rel="icon" href="https://img.freepik.com/premium-vector/medical-care-health-care-icon-concept_313674-557.jpg">
    <link rel="stylesheet" href="{{ url_for('static', filename='login.css') }}">
</head>
<body>
    <div class="container">
        <div class="div1">
            <h1>MedAnalyzer System</h1>
            <form action="" method="post" id="loginForm" class="fade-in">

```

```

<label for="email">Email :</label>

<div class="input-wrapper">
  <i class="fas fa-envelope icon-left"></i>
  <input type="email" name="email" id="email" placeholder="Email" required>
</div>

<label for="password">Password :</label>
<div class="input-wrapper">
  <i class="fas fa-lock icon-left"></i>
  <input type="password" id="password" name="password" placeholder="password" required>
  <i class="fas fa-eye icon-right" id="togglePassword" onclick="togglePassword()"></i>
</div>

<label for="role"> Role :</label>
<select id="role" class="user_type" name="role" required onchange="handleRoleChange()">
  <option value="" disabled selected>Select Role</option>
  <option value="doctor">Doctor</option>
  <option value="nurse">Nurse</option>
  <option value="patient">Patient</option>
</select>

<!-- Patient الذي يظهر فقط إذا كان الدور patient ID حقل -->
<div id="patientIdField" style="display: none;">
  <label for="patient_id">Patient ID (optional):</label>
  <div class="input-wrapper">
    <i class="fas fa-id-card icon-left"></i>
    <input type="text" name="patient_id" id="patient_id" placeholder="Enter your Patient ID">
  </div>
</div>

<p class="p1">
  <a href="{{ url_for('forgot_password') }}" class="a1">forgot password?</a>
</p>

<button type="submit" id="btn">LOGIN</button>

<p class="error">all fields are required</p>

<div id="login-error" class="error-message" style="display: none;"></div>
</form>

<p class="create-accaount">Don't have an account ? <a href="{{ url_for('register') }}">Create an account</a></p>
</div>
</div>

<script>
function handleRoleChange() {
  const roleSelect = document.getElementById('role');
  const patientIdField = document.getElementById('patientIdField');
  if (roleSelect.value === 'patient') {
    patientIdField.style.display = 'block';
  } else {
    patientIdField.style.display = 'none';
    // تنظيف الحقل إذا لم يكن الدور
    document.getElementById('patient_id').value = "";
  }
}

```

```

// لتشغيل الدالة عند تحميل الصفحة إذا كانت القيمة محددة مسبقًا
document.addEventListener('DOMContentLoaded', () => {
    handleRoleChange();
});

// دالة لتبديل عرض كلمة المرور (اختياري)
function togglePassword() {
    const passwordInput = document.getElementById('password');
    const toggleIcon = document.getElementById('togglePassword');
    if (passwordInput.type === 'password') {
        passwordInput.type = 'text';
        toggleIcon.classList.remove('fa-eye');
        toggleIcon.classList.add('fa-eye-slash');
    } else {
        passwordInput.type = 'password';
        toggleIcon.classList.remove('fa-eye-slash');
        toggleIcon.classList.add('fa-eye');
    }
}
</script>

</body>
</html>

Code 3(Javascript for login page)
// Function to validate email format
function validateEmail(email) {
    const emailPattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
    return emailPattern.test(email);
}

// Function to validate password
function validatePassword(password) {
    // Ensure the password includes at least 6 characters
    return password.length >= 6;
}

// Function to toggle show/hide password
function togglePassword() {
    const passwordInput = document.getElementById("password");
    const passwordIcon = document.querySelector(".icon-right");

    if (passwordInput.type === "password") {
        passwordInput.type = "text";
        passwordIcon.classList.remove("fa-eye-slash");
        passwordIcon.classList.add("fa-eye");
    } else {
        passwordInput.type = "password";
        passwordIcon.classList.remove("fa-eye");
        passwordIcon.classList.add("fa-eye-slash");
    }
}

// Add event listener for the login form submission
document.getElementById("loginform").addEventListener("submit", function(event) {
    event.preventDefault(); // Prevent default form submission

    const email = document.getElementById("email").value;
    const password = document.getElementById("password").value;

```

```

const role = document.getElementById("role").value; // Get selected role
const errorMessage = document.querySelector(".error");

// Input validation
if (!email || !password || !role) {
  errorMessage.textContent = "All fields are required!";
  errorMessage.classList.add("show");

  setTimeout(() => {
    errorMessage.classList.remove("show");
  }, 5000);

} else if (!validateEmail(email)) {
  errorMessage.textContent = "Please enter a valid email address.";
  errorMessage.classList.add("show");

  setTimeout(() => {
    errorMessage.classList.remove("show");
  }, 5000);

} else if (!validatePassword(password)) {
  errorMessage.textContent = "Password must be at least 6 characters long.";
  errorMessage.classList.add("show");

  setTimeout(() => {
    errorMessage.classList.remove("show");
  }, 5000);

} else {
  errorMessage.classList.remove("show");

  // Prepare form data and include role
  const formData = new FormData();
  formData.append("email", email);
  formData.append("password", password);
  formData.append("role", role);

  // Send data to the backend using fetch
  fetch('/login', {
    method: 'POST',
    body: formData
  })
  .then(response => response.json())
  .then(data => {
    if (data.success) {
      alert("Login successful! Redirecting to dashboard...");

      // Store role and email in localStorage
      localStorage.setItem("role", role);
      localStorage.setItem("email", email);

      // Redirect based on role
      if (role.toLowerCase() === 'doctor' || role.toLowerCase() === 'nurse') {
        window.location.href = "Dashboard.html";
      } else if (role.toLowerCase() === 'patient') {
        window.location.href = "Analytics_Dashboard.html";
      } else {
        alert("Unknown user role! Redirecting to login.");
        window.location.href = "login.html";
      }
    }
  });
}

```

```

    }

    } else {
        // If login failed, show error message
        errorMessage.textContent = data.message || "Invalid email or password!";
        errorMessage.classList.add("show");

        setTimeout(() => {
            errorMessage.classList.remove("show");
        }, 5000);
    }
})
.catch(error => {
    // Handle connection errors
    console.error("Error:", error);
    errorMessage.textContent = "An error occurred. Please try again later.";
    errorMessage.classList.add("show");

    setTimeout(() => {
        errorMessage.classList.remove("show");
    }, 5000);
});
}
});

```

Code 4(HTML for create account page)

```

<!DOCTYPE html>
<html lang="en">
<head>
    <title>Create Account - MedAnalyzer System</title>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />

    <link rel="icon" href="https://img.freepik.com/free-vector/add-new-user_78370-4710.jpg" />

    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
href="https://fonts.googleapis.com/css2?family=Abril+Fatface&family=Caveat:wght@400..700&family=Oswald:wght@200..700&display=swap" rel="stylesheet" />

    <link rel="stylesheet" href="{{ url_for('static', filename='create_account.css') }}" />
    <script src="{{ url_for('static', filename='main.js') }}" defer></script>
    <script src="{{ url_for('static', filename='create_account.js') }}" defer></script>
</head>
<body>
    <div class="container">

        <h1>Create Your Account</h1>
        <p class="p1">Please fill out the information below to create an account for accessing the
MedAnalyzer system.</p>

        <form action="{{ url_for('register') }}" method="post">

```

```

<label for="email">Email:</label>
<input type="email" id="email" name="email" placeholder="Enter your email" required />

<label for="password">Password:</label>
<input type="password" id="password" name="password" placeholder="Enter your
password" required />

<label for="confirm_password">Confirm Password:</label>
<input type="password" id="confirm_password" name="confirm_password"
placeholder="Confirm your password" required />

<label for="role">Role:</label>
<select id="role" name="role" required>
  <option value="doctor">Doctor</option>
  <option value="nurse">Nurse</option>
  <option value="patient">Patient</option>
</select>

<button type="submit" id="btn">Create Account</button>
</form>

<p class="success-message" id="success-message" style="display:none;">
  Your account has been created successfully!
</p>

<a href="{{ url_for('login') }}">Back To Login</a>
{% with messages = get_flashed_messages(with_categories=true) %}
  {% if messages %}
    <div>
      {% for category, message in messages %}
        <div class="alert alert-{{ category }}">
          {{ message }}
        </div>
      {% endfor %}
    </div>
  {% endif %}
{% endwith %}
</div>
</body>
</html>

```

Code 5 (java script for create account page)

```

window.onload = function() {
  // Get DOM elements
  const form = document.querySelector('form');
  const emailField = document.getElementById('email');
  const passwordField = document.getElementById('password');
  const confirmPasswordField = document.getElementById('confirm-password');
  const roleField = document.getElementById('role');
  const successMessage = document.getElementById('success-message');

  // Function to validate email format

```

```

function isValidEmail(email) {
  const re = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
  return re.test(email);
}

// Ensure required elements exist
if (!form || !emailField || !passwordField || !confirmPasswordField || !roleField) {
  console.error("Some required form elements are missing!");
  return; // Do not proceed if elements are missing
}

// Form submission event
form.onSubmit = async function(event) {
  // Prevent default form submission
  event.preventDefault();

  // Get trimmed values
  const email = emailField.value.trim();
  const password = passwordField.value.trim();
  const confirmPassword = confirmPasswordField.value.trim();
  const role = document.getElementById('role').value;

  // Validate email format
  if (!isValidEmail(email)) {
    alert("Please enter a valid email address.");
    return;
  }

  // Check if role is selected
  if (!role) {
    alert("Please select a role.");
    return;
  }

  // Check if passwords match
  if (password !== confirmPassword) {
    alert("Passwords do not match!");
    return;
  }

  // Ensure password is not too short
  if (password.length < 6) {
    alert("Password must be at least 6 characters long!");
    return;
  }

  // Create data object to send to the backend
  const formData = {
    email: email,
    password: password,
    role: role,
  };

  try {
    // Send data to the server using fetch
    const response = await fetch('/api/create-account', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
    });
  }

```

```

    },
    body: JSON.stringify(formData)
  });

const data = await response.json();

if (response.ok) {
  // If successful, show success message
  successMessage.style.display = 'block';
  successMessage.textContent = "Your account has been created successfully!";

  // Reset form fields
  form.reset();

  // Store user information in localStorage
  localStorage.setItem("userEmail", email);
  localStorage.setItem("role", role);

  // Redirect based on user role
  if (role === "doctor" || role === "nurse") {
    window.location.href = "Dashboard.html";
  } else if (role === "patient") {
    window.location.href = "Analytics_Dashboard.html";
  }

} else {
  // Show error message from server
  alert(data.message || "An error occurred while creating your account.");
}

} catch (error) {
  // Show error if fetch fails
  alert("An error occurred. Please try again.");
}
};
};

```

Code 6(HTML for forgotpassword page)

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Forgot Password</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='forgot_password.css') }}">
  <script src="{{ url_for('static', filename='forgot_password.js') }}" defer></script>
</head>
<body>
  <h2>Reset Your Password</h2>
  <form id="reset-form">
    <input type="email" id="email" placeholder="Enter your email" required><br>
    <input type="password" id="new_password" placeholder="Enter new password" required><br>
    <button type="submit">Reset Password</button>
  </form>

  <p id="success-message" style="display:none;"></p>
  <p>Remembered your password? <a href="{{ url_for('login') }}">Login here</a></p>
</body>
</html>

```

Code 7(javascript for forgotpassword)

```
document.addEventListener("DOMContentLoaded", function () {
    const form = document.getElementById("reset-form");
    const emailInput = document.getElementById("email");
    const passwordInput = document.getElementById("new_password");
    const successMessage = document.getElementById("success-message");

    form.addEventListener("submit", function (event) {
        event.preventDefault();

        const email = emailInput.value.trim();
        const newPassword = passwordInput.value.trim();

        if (!email || !newPassword) {
            alert("Please enter your email and new password.");
            return;
        }

        fetch("/forgot_password", {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify({
                email: email,
                new_password: newPassword
            })
        })
        .then(response => response.json())
        .then(data => {
            if (data.success) {
                successMessage.style.display = "block";
                successMessage.textContent = data.message;
            } else {
                alert("Error: " + data.message);
            }
        })
        .catch(error => {
            alert("An error occurred: " + error);
            console.error(error);
        });
    });
});
```

Code 8(HTML for home page)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Medical Data Platform</title>
<script src="{ { url_for('static', filename='main.js') } }" defer></script>
<script src="{ { url_for('static', filename='Dashboard.js') } }" defer></script>
<link rel="stylesheet" href="{ { url_for('static', filename='Dashboard.css') } }">
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome/6.5.0/css/all.min.css" />
</head>
<body>

<header>
<div class="logo">MedAnalyzer System</div>
<!-- نموذج البحث -->
```

```

<form id="search-form">
  <input type="text" id="patient_id" placeholder="Enter Patient ID" required>
  <button type="submit">Search</button>
</form>

<!-- مكان عرض نتائج المريض -->
<div id="search-result"></div>

<div id="statusMessage" style="color: red; margin-top: 10px;"></div>

<!-- زر تسجيل الخروج باستخدام POST -->
<form action="/logout" method="POST" style="display: inline;">
  <button type="submit" class="logout-btn">Logout</button>
</form>

</header>

<main>
  <div class="card-container">
    <a href="/Doctors_page" class="card" style="background-image:
url('https://i.ibb.co/Q3GRQNQV/Doctors.jpg');">Doctors platform</a>
    <a href="/patient_data_entry" class="card" style="background-image: url('https://i.ibb.co/dJbNdqwG/Analytic-
data.jpg');">Patient Data Entry</a>
    <a href="/Analytics" class="card" style="background-image: url('https://i.ibb.co/G3xVqPWB/photo-2025-05-23-12-23-
26.jpg');">Analytic-data</a>
    <a href="/about" class="card" style="background-image: url('https://i.ibb.co/23rxsPCk/about.jpg');">About</a>
  </div>
</main>

<footer>
  © 2025 MedAnalyzer. All rights reserved.
</footer>

<script>
  const searchForm = document.getElementById("search-form");
  const resultDiv = document.getElementById("search-result");

  if (searchForm && resultDiv) {
    searchForm.addEventListener("submit", function (e) {
      e.preventDefault();

      const PatientID = document.getElementById("patient_id").value;

      fetch("/search_patient", {
        method: "POST",
        headers: {
          "Content-Type": "application/x-www-form-urlencoded"
        },
        body: new URLSearchParams({ PatientID: PatientID })
      })
      .then(response => response.json())
      .then(data => {
        console.log("📄 Response Data:", data);

        if (data.found) {
          const latest = data.latest_info;
          const readings = data.all_readings;
          const medicines = data.all_medicines;
          const histories = data.all_histories;

```

```

let html = `
  <h3>📄 Latest Patient Info</h3>
  <ul>
    <li><strong>PatientID:</strong> ${latest.PatientID}</li>
    <li><strong>Age:</strong> ${latest.Age}</li>
    <li><strong>Gender:</strong> ${latest.Gender}</li>
    <li><strong>Weight:</strong> ${latest.Weight_kg ?? "N/A"} kg</li>
    <li><strong>Height:</strong> ${latest.Height_m ?? "N/A"} m</li>
    <li><strong>BMI:</strong> ${latest.Derived_BMI}</li>
    <li><strong>MAP:</strong> ${latest.Derived_MAP}</li>
    <li><strong>Pulse Pressure:</strong> ${latest.Derived_Pulse_Pressure}</li>
    <li><strong>HRV:</strong> ${latest.Derived_HRV}</li>
  </ul>

  <h4>💊 Medicines:</h4>
  <ul>${(medicines && medicines.length > 0) ? medicines.map(m => `<li>${m}</li>`).join("") :
"<li>N/A</li>"}</ul>

  <h4>📖 Medical History:</h4>
  <ul>${(histories && histories.length > 0) ? histories.map(h => `<li>${h}</li>`).join("") :
"<li>N/A</li>"}</ul>

  <h4>📊 All Vital Readings:</h4>
  <div style="max-width: 550px; overflow-x: auto; margin-top: 7px;">
  <table border="1" style="border-collapse: collapse; width: 100%;">
    <thead>
      <tr>
        <th>Timestamp</th>
        <th>Heart Rate</th>
        <th>Respiratory Rate</th>
        <th>Temperature</th>
        <th>Oxygen Saturation</th>
        <th>Systolic BP</th>
        <th>Diastolic BP</th>
      </tr>
    </thead>
    <tbody>
      ${
        readings.map(r => `
          <tr>
            <td>${r.Timestamp}</td>
            <td>${r.HeartRate}</td>
            <td>${r.RespiratoryRate}</td>
            <td>${r.BodyTemperature}</td>
            <td>${r.OxygenSaturation}</td>
            <td>${r.SystolicBloodPressure}</td>
            <td>${r.DiastolicBloodPressure}</td>
          </tr>
        `).join("")
      }
    </tbody>
  </table>
`;

resultDiv.innerHTML = html;
} else {
  resultDiv.innerHTML = `<p style="color: red;">❌ No patient found with the entered ID.</p>`;
}

```

```

    })
    .catch(error => {
      console.error("✖ Error:", error);
      resultDiv.innerHTML = `<p style="color: red;">✖ An error occurred: ${error.message}</p>`;
    });
  });
}
</script>

```

</body>

</html>

Code 9(javascript for Dashboard page)

// التحقق من صلاحية الدور قبل تحميل أي شيء

```

const role = localStorage.getItem("role");
if (role && role.toLowerCase() === "patient") {
  alert("Access denied. Redirecting to analytics page.");
  window.location.href = "Analytics_Dashboard.html";
}

```

// زر تسجيل الخروج

```

const logoutBtn = document.getElementById("logout-Btn");
logoutBtn.addEventListener("click", () => {
  // يمكنك هنا مسح بيانات الجلسة أو التوجيه لصفحة تسجيل الدخول
  alert("You have been logged out.");
  window.location.href = "login.html";
});

```

```

const searchInput = document.getElementById("input_with_icon");
const patientDetailsDiv = document.getElementById("patient-details");
let patientsData = [];

```

// عند بداية الصفحة CSV تحميل ملف

```

fetch("human_vital_signs_dataset_2024.csv")
  .then(response => response.text())
  .then(csvText => {
    Papa.parse(csvText, {
      header: true,
      skipEmptyLines: true,
      complete: function (results) {
        patientsData = results.data;
      },
    });
  });

```

// فقط Enter البحث عند الضغط على

```

searchInput.addEventListener("keydown", function (event) {
  if (event.key === "Enter") {
    const searchValue = searchInput.value.trim();
    if (searchValue !== "") {
      const patient = patientsData.find(p => p["Patient ID"] === searchValue);

      if (patient) {
        const { Age, Gender, ["Weight (kg)": Weight, ["Height (m)": Height, ["Patient ID": ID } = patient;

        patientDetailsDiv.innerHTML = `
          <div class="patient-box">
            <h3>Patient Information</h3>
            <p><strong>ID:</strong> ${ID}</p>
            <p><strong>Age:</strong> ${Age}</p>
            <p><strong>Gender:</strong> ${Gender}</p>
          </div>
        `;
      }
    }
  }
});

```

```

        <p><strong>Weight:</strong> ${parseFloat(Weight).toFixed(1)} kg</p>
        <p><strong>Height:</strong> ${parseFloat(Height).toFixed(2)} m</p>
    </div>
    `;
    patientDetailsDiv.style.display = "block";
} else {
    patientDetailsDiv.innerHTML = "<p>No patient found with this ID.</p>";
    patientDetailsDiv.style.display = "block";
}
} else {
    patientDetailsDiv.style.display = "none";
}
}
});
document.addEventListener("click", function (e) {
    const isClickInside =
        patientDetailsDiv.contains(e.target) || searchInput.contains(e.target);
    if (!isClickInside) {
        patientDetailsDiv.style.display = "none";
    }
});

```

Code 10 (HTML fore data entry page)

```

<!DOCTYPE html>
<html lang="en">
<head>
    <title>Patient Data Entry - MedAnalyzer System</title>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link rel="icon" href="https://thumbs.dreamstime.com/b/data-entry-shiny-blue-square-button-isolated-143172690.jpg">
    <link rel="stylesheet" href="{{ url_for('static', filename='patient_data_entry.css') }}">
    <script src="{{ url_for('static', filename='save_patient.js') }}" defer></script>
    <script src="{{ url_for('static', filename='patient_data_entry.js') }}" defer></script>
    <script src="{{ url_for('static', filename='main.js') }}" defer></script>
    <script src="{{ url_for('static', filename='upload.js') }}" defer></script>
</head>
<body>
    <div class="page-title">
        <h1>Enter Patient Data</h1>
    </div>
    <form id="patient-form">
        <label for="PatientID">Patient ID :</label>
        <input type="text" id="PatientID" name="PatientID" required>

        <label for="HeartRate">Heart Rate :</label>
        <input type="number" id="HeartRate" name="HeartRate" min="1" required>

        <label for="RespiratoryRate">Respiratory Rate :</label>
        <input type="number" id="RespiratoryRate" name="RespiratoryRate" min="5" max="60" required>

        <label for="Timestamp">Timestamp :</label>
        <input type="datetime-local" id="Timestamp" name="Timestamp" step="1" required>

        <label for="BodyTemperature">Body Temperature :</label>
        <input type="number" id="BodyTemperature" name="BodyTemperature" step="any" min="30" max="45" required>

        <label for="OxygenSaturation">Oxygen Saturation :</label>
        <input type="number" id="OxygenSaturation" name="OxygenSaturation" step="any" min="50" max="100" required>
    </form>

```

```

<label for="SystolicBloodPressure">Systolic Blood Pressure :</label>
<input type="number" id="SystolicBloodPressure" name="SystolicBloodPressure" step="1" min="50" max="250"
required>

<label for="DiastolicBloodPressure">Diastolic Blood Pressure :</label>
<input type="number" id="DiastolicBloodPressure" name="DiastolicBloodPressure" step="1" min="30" max="150"
required>

<label for="Age">Age :</label>
<input type="number" id="Age" name="Age" step="1" min="0" max="120" required>

<label for="Gender">Gender :</label>
<select id="Gender" name="Gender" required>
  <option value="" disabled selected>Select Gender</option>
  <option value="Male">Male</option>
  <option value="Female">Female</option>
</select>

<label for="Weight_kg">Weight (kg) :</label>
<input type="number" id="Weight_kg" name="Weight_kg" step="0.01" min="1" max="300" required>

<label for="Height_m">Height (m) :</label>
<input type="number" id="Height_m" name="Height_m" step="0.01" min="0.3" max="2.5" required>

<button type="button" onclick="calculateBMI()" class="btn-bmi">Calculate BMI</button>
<button type="button" onclick="calculatePulsePressure()">Calculate Pulse Pressure</button>
<button type="button" onclick="calculateMAP()">Calculate MAP</button>
<button type="button" onclick="calculateHRV()">Calculate HRV</button>

<br>

<label for="Derived_HRV">HRV :</label>
<input type="text" id="Derived_HRV" name="Derived_HRV" readonly>

<label for="Derived_Pulse_Pressure">Pulse Pressure :</label>
<input type="text" id="Derived_Pulse_Pressure" name="Derived_Pulse_Pressure" readonly>

<label for="Derived_BMI">BMI :</label>
<input type="text" id="Derived_BMI" name="Derived_BMI" readonly>

<label for="Derived_MAP">MAP :</label>
<input type="text" id="Derived_MAP" name="Derived_MAP" readonly>
<hr>

<div class="custom-sections">

<div class="section">
  <label for="medicine">☐ Currently and Previously on [Medications]:</label>
  <textarea id="medicine" name="medicine" rows="3" placeholder="Enter the name of the medicine here..."></textarea>
</div>

<label>📋 Patient History:</label>
<div id="patient_history_checklist" class="checkbox-list">
  <label><input type="checkbox" name="patient_history" value="Hypertension"> Hypertension</label>
  <label><input type="checkbox" name="patient_history" value="Diabetes"> Diabetes</label>
  <label><input type="checkbox" name="patient_history" value="Asthma"> Asthma</label>
  <label><input type="checkbox" name="patient_history" value="Heart Disease"> Heart Disease</label>
  <label><input type="checkbox" name="patient_history" value="Kidney Disease"> Kidney Disease</label>
  <label><input type="checkbox" name="patient_history" value="Cancer"> Cancer</label>

```

```

<label><input type="checkbox" name="patient_history" value="COVID-19"> COVID-19</label>

<label>
  <input type="checkbox" name="patient_history" value="Other" id="other_checkbox"> Other
</label>

<!-- Hidden input for 'Other' text -->
<label id="other-input-label" style="display:none;">
  Please specify:
  <input type="text" id="other_history_input" name="other_history_input">
</label>
</div>

</div>

  <div id="alerts"></div>
  <button type="submit" class="btn-save">Save Patient Data</button>
</form>

<fieldset>
  <legend>CSV/Excel file</legend>
  <input type="file" id="file-upload" name="file-upload" />
  <button type="button" onclick="uploadfile()">Upload</button>
  <p id="warningMessage" style="color: red; display: none;"></p>

</fieldset>

<footer class="footer">
  <p>&copy; 2025 MedAnalyzer System</p>
</footer>
<script>
document.addEventListener("DOMContentLoaded", function () {
  const otherCheckbox = document.getElementById("other_checkbox");
  const otherInputLabel = document.getElementById("other-input-label");

  if (otherCheckbox) {
    otherCheckbox.addEventListener("change", function () {
      otherInputLabel.style.display = this.checked ? "block" : "none";
    });
  }
});
</script>

</body>
</html>
Code 11(javascript for data entry page)
function markInvalidField(input) {
  input.classList.add("invalid-input");
}

function clearFieldHighlights() {
  document.querySelectorAll(".invalid-input").forEach(input => {
    input.classList.remove("invalid-input");
  });
}

function calculateBMI() {
  clearFieldHighlights();

```

```

const weightEl = document.getElementById("Weight_kg");
const heightEl = document.getElementById("Height_m");
const weight = parseFloat(weightEl.value);
const height = parseFloat(heightEl.value);

if (!weight || !height || weight <= 0 || height <= 0) {
  alert("Please enter valid weight and height to calculate BMI.");
  if (!weight || weight <= 0) markInvalidField(weightEl);
  if (!height || height <= 0) markInvalidField(heightEl);
  return;
}

const bmi = weight / (height * height);
document.getElementById("Derived_BMI").value = bmi.toFixed(2);
showAlert("bmi", bmi);
}

function calculatePulsePressure() {
  clearFieldHighlights();
  const systolicEl = document.getElementById("SystolicBloodPressure");
  const diastolicEl = document.getElementById("DiastolicBloodPressure");
  const systolic = parseFloat(systolicEl.value);
  const diastolic = parseFloat(diastolicEl.value);

  if (!systolic || !diastolic || systolic <= 0 || diastolic <= 0) {
    alert("Please enter valid systolic and diastolic pressure to calculate Pulse Pressure.");
    if (!systolic || systolic <= 0) markInvalidField(systolicEl);
    if (!diastolic || diastolic <= 0) markInvalidField(diastolicEl);
    return;
  }

  const pp = systolic - diastolic;
  document.getElementById("Derived_Pulse_Pressure").value = pp.toFixed(2);
  showAlert("pp", pp);
}

function calculateMAP() {
  clearFieldHighlights();
  const systolicEl = document.getElementById("SystolicBloodPressure");
  const diastolicEl = document.getElementById("DiastolicBloodPressure");
  const systolic = parseFloat(systolicEl.value);
  const diastolic = parseFloat(diastolicEl.value);

  if (!systolic || !diastolic || systolic <= 0 || diastolic <= 0) {
    alert("Please enter valid systolic and diastolic pressure to calculate MAP.");
    if (!systolic || systolic <= 0) markInvalidField(systolicEl);
    if (!diastolic || diastolic <= 0) markInvalidField(diastolicEl);
    return;
  }

  const map = (systolic + 2 * diastolic) / 3;
  document.getElementById("Derived_MAP").value = map.toFixed(2);
  showAlert("map", map);
}

function calculateHRV() {
  clearFieldHighlights();
  const heartRateEl = document.getElementById("HeartRate");
  const heartRate = parseFloat(heartRateEl.value);

```

```

if (!heartRate || heartRate <= 0) {
  alert("Please enter a valid heart rate to calculate HRV.");
  markInvalidField(heartRateEl);
  return;
}

const hrv = (60000 / heartRate);
document.getElementById("Derived_HRV").value = hrv.toFixed(2);
showAlert("hrv", hrv);
}

function showAlert(type, value) {
  const alertBox = document.getElementById("alerts");
  let message = "";
  let isAbnormal = false;

  switch (type) {
    case "bmi":
      if (value < 18.5 || value > 25) {
        message = `Abnormal BMI: ${value.toFixed(2)}`;
        isAbnormal = true;
      }
      break;
    case "pp":
      if (value < 30 || value > 60) {
        message = `Abnormal Pulse Pressure: ${value.toFixed(2)} mmHg`;
        isAbnormal = true;
      }
      break;
    case "map":
      if (value < 65 || value > 105) {
        message = `Abnormal MAP: ${value.toFixed(2)} mmHg`;
        isAbnormal = true;
      }
      break;
    case "hrv":
      if (value < 400 || value > 1200) {
        message = `Abnormal HRV: ${value.toFixed(2)} ms`;
        isAbnormal = true;
      }
      break;
  }

  alertBox.textContent = isAbnormal ? message : "";
}

document.querySelector(".btn-save").addEventListener("click", async function () {
  clearFieldHighlights();
  const form = document.querySelector("form");
  const formData = new FormData(form);
  const jsonData = {};
  let hasEmptyField = false;
  let hasInvalidNumber = false;

  formData.forEach((value, key) => {
    const input = form.querySelector(`[name="${key}"]`);
    if (!value.trim()) {
      hasEmptyField = true;
    }
  });
});

```

```

    markInvalidField(input);
  } else if (input && input.type === "number" && isNaN(parseFloat(value))) {
    hasInvalidNumber = true;
    markInvalidField(input);
  }
  jsonData[key] = value;
});

if (hasEmptyField || hasInvalidNumber) {
  let message = "Please check the following:\n";
  if (hasEmptyField) message += "- All fields must be filled.\n";
  if (hasInvalidNumber) message += "- Some numeric fields are invalid.\n";
  alert(message);
  return;
}

try {
  const response = await fetch("/patient_data_entry", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(jsonData)
  });

  if (response.ok) {
    alert("Data saved successfully");
  } else {
    alert("Failed to save data");
  }
} catch (error) {
  alert("Error connecting to the server");
  console.error(error);
}
});

async function uploadfile() {
  const fileInput = document.getElementById("file-upload");
  const file = fileInput.files[0];
  const warningMessage = document.getElementById("warningMessage");

  if (!file) {
    warningMessage.style.display = "block";
    warningMessage.textContent = "Please select a file to upload";
    return;
  }

  const allowedTypes = [
    "text/csv",
    "application/vnd.ms-excel",
    "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
  ];

  if (!allowedTypes.includes(file.type)) {
    warningMessage.style.display = "block";
    warningMessage.textContent = "Unsupported file format";
    return;
  }

  warningMessage.style.display = "none";

```

```

try {
  const formData = new FormData();
  formData.append("csvFile", file);

  const response = await fetch("https://your-server.com/api/upload-csv", {
    method: "POST",
    body: formData
  });

  if (response.ok) {
    alert("File uploaded successfully!");
  } else {
    alert("Failed to upload the file.");
  }
} catch (error) {
  alert("Error occurred during file upload.");
  console.error(error);
}
}

```

Code 12(JavaScript for save patient data on database)

```

document.getElementById("patient-form").addEventListener("submit", async function (e) {
  e.preventDefault();
  const BASE_URL = 'http://127.0.0.1:5000';
  const form = e.target;
  const formData = new FormData(form);
  const params = new URLSearchParams();

  for (const [key, value] of formData.entries()) {
    params.append(key, value);
  }

```

```

try {
  const response = await fetch("/save_patient", {
    method: "POST",
    headers: { "Content-Type": "application/x-www-form-urlencoded" },
    body: params
  });

  const resultBox = document.getElementById("alerts");
  if (response.ok) {
    resultBox.style.color = "green";
    resultBox.textContent = "✔ Patient data saved successfully";
    form.reset(); // Reset the form
  } else {
    resultBox.style.color = "red";
    resultBox.textContent = "✗ Failed to save patient data";
  }
} catch (err) {
  console.error("✗ Error during submission:", err);
  document.getElementById("alerts").style.color = "red";
  document.getElementById("alerts").textContent = "✗ Server connection error";
}
});

```

Code 13(HTML for doctors plat form)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <title>Doctors page-MedAnalyzer System</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">

```

```

<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link
href="https://fonts.googleapis.com/css2?family=Abril+Fatface&family=Caveat:wght@400..700&family=Oswald:wght@200..700&display=swap" rel="stylesheet">
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="icon" href="https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcRg1OpU3p4Qch47iqaHd7oNLgEE8zyY7QvE2A&s">
<link rel="stylesheet" href="{{ url_for('static', filename='Doctors_page.css') }}">
<script src="{{ url_for('static', filename='main.js') }}" defer></script>
<script src="{{ url_for('static', filename='Doctors_page.js') }}" defer></script>
</head>
<body>
<h1>Platform Doctors</h1>
<div class="doctors-container">
  <div class="doctor-card" id="card1">
    
    <h2>Dr. Mohamed Ahmed</h2>
    <p>Cardiologist</p>
    <p>20001</p>
    <button onclick="toggleCard('card1')">Click to view details</button>
    <div class="Additional-details">
      <p><strong>Phone:</strong> 0791234567</p>
      <p><strong>Email:</strong> mohamed.ahmed@clinic.com</p>
      <p><strong>Experience:</strong> 10 years</p>
      <p><strong>Work Hours:</strong> Sunday to Thursday, 9:00 AM - 5:00 PM</p>
    </div>
  </div>
  <div class="doctor-card" id="card2">
    
    <h2>Dr. Sarah Ali</h2>
    <p>Neurologist</p>
    <p>20002</p>
    <button onclick="toggleCard('card2')">Click to view details</button>
    <div class="Additional-details">
      <p><strong>Phone:</strong>0792345678</p>
      <p><strong>Email:</strong> sarah.ali@clinic.com</p>
      <p><strong>Experience:</strong> 8 years</p>
      <p><strong>Work Hours:</strong> Monday to Friday, 10:00 AM - 6:00 PM</p>
    </div>
  </div>
  <div class="doctor-card" id="card3">
    
    <h2>Dr. Khaled Mansour</h2>
    <p>Pediatrician</p>
    <p>20003</p>
    <button onclick="toggleCard('card3')">Click to view details</button>
    <div class="Additional-details">
      <p><strong>Phone:</strong>0793456789</p>
      <p><strong>Email:</strong> khaled.mansour@clinic.com</p>
      <p><strong>Experience:</strong> 12 years</p>
      <p><strong>Work Hours:</strong> Sunday to Thursday, 8:30 AM - 4:30 PM</p>
    </div>
  </div>
  <div class="doctor-card" id="card4">

```

src="https://th.bing.com/th/id/OIP.fuAk5xXWjuQXJzIH1780bQHaHa?w=204&h=204&c=7&r=0&o=5&dpr=1.3&pid=1.7" alt="Doctor 4">

<h2>Dr. Asmaa Ali</h2>

<p>Pharmacist</p>

<p>20004</p>

<button onclick="toggleCard('card4')">Click to view details</button>

<div class="Additional-details">

<p>Phone:0794567890</p>

<p>Email: asmaa.ali@clinic.com</p>

<p>Experience: 5 years</p>

<p>Work Hours: Sunday to Thursday, 9:00 AM - 5:00 PM</p>

</div>

</div>

<div class="doctor-card" id="card5">

src="https://th.bing.com/th/id/OIP.B0yfMsiXhRUwJqn8BADPzwHaHa?w=2000&h=2000&rs=1&pid=ImgDetMain" alt="Doctor 4">

<h2>Dr. Sora Ali</h2>

<p>Cardiologist</p>

<p>20005</p>

<button onclick="toggleCard('card5')">Click to view details</button>

<div class="Additional-details">

<p>Phone:0795678901</p>

<p>Email: sora.ali@clinic.com</p>

<p>Experience: 15 years</p>

<p>Work Hours: Monday to Friday, 11:00 AM - 7:00 PM</p>

</div>

</div>

<div class="doctor-card" id="card6">

<h2>Dr. Tamara Mwafaq</h2>

<p>Neurologist</p>

<p>20006</p>

<button onclick="toggleCard('card6')">Click to view details</button>

<div class="Additional-details">

<p>Phone:0781234567</p>

<p>Email: tamara.mwafaq@clinic.com</p>

<p>Experience: 7 years</p>

<p>Work Hours: Sunday to Thursday, 10:00 AM - 6:00 PM</p>

</div>

</div>

<div class="doctor-card" id="card7">

src="https://th.bing.com/th/id/OIP.psWM5EBVvNzggzXZ_qQNggHaHa?w=626&h=626&rs=1&pid=ImgDetMain" alt="Doctor 4">

<h2>Dr. Amal Ayman</h2>

<p>Pediatrician</p>

<p>20007</p>

<button onclick="toggleCard('card7')">Click to view details</button>

<div class="Additional-details">

<p>Phone:0782345678</p>

<p>Email: amal.ayman@clinic.com</p>

<p>Experience: 9 years</p>

<p>Work Hours: Sunday to Thursday, 8:00 AM - 4:00 PM</p>

</div>

</div>

<div class="doctor-card" id="card8">

```

        
        <h2>Dr. Deema Alkhalidi</h2>
        <p>Family Medicine Physician</p>
        <p>20008</p>
        <button onclick="toggleCard('card8')">Click to view details</button>
        <div class="Additional-details">
            <p><strong>Phone:</strong>0783456789</p>
            <p><strong>Email:</strong> deema.alkhalidi@clinic.com</p>
            <p><strong>Experience:</strong> 4 years</p>
            <p><strong>Work Hours:</strong> Monday to Saturday, 9:30 AM - 5:30 PM</p>
        </div>
    </div>
    <div class="doctor-card" id="card9">
        
        <h2>Dr.Ali Ahmad</h2>
        <p>Dermatologist</p>
        <p>20009</p>
        <button onclick="toggleCard('card9')">Click to view details</button>
        <div class="Additional-details">
            <p><strong>Phone:</strong>0784567890</p>
            <p><strong>Email:</strong> ali.ahmad@clinic.com</p>
            <p><strong>Experience:</strong> 11 years</p>
            <p><strong>Work Hours:</strong> Sunday to Thursday, 9:00 AM - 3:00 PM</p>
        </div>
    </div>
    <div class="doctor-card" id="card10">
        
        <h2>Dr. Alaa Youssef</h2>
        <p>Pulmonologist</p>
        <p>200010</p>
        <button onclick="toggleCard('card10')">Click to view details</button>
        <div class="Additional-details">
            <p><strong>Phone:</strong>0785678901</p>
            <p><strong>Email:</strong> alaa.youssef@clinic.com</p>
            <p><strong>Experience:</strong> 6 years</p>
            <p><strong>Work Hours:</strong> Monday to Friday, 10:00 AM - 6:00 PM</p>
        </div>
    </div>
    <div class="doctor-card" id="card11">
        
        <h2>Dr. Youssef Ameen</h2>
        <p>Oncologist</p>
        <p>200011</p>
        <button onclick="toggleCard('card11')">Click to view details</button>
        <div class="Additional-details">
            <p><strong>Phone:</strong>0771234567</p>
            <p><strong>Email:</strong> youssef.ameen@clinic.com</p>
            <p><strong>Experience:</strong> 14 years</p>
            <p><strong>Work Hours:</strong> Sunday to Thursday, 9:00 AM - 5:00 PM</p>
        </div>
    </div>
    <div class="doctor-card" id="card12">

```


<h2>Dr. Khalid Ahmad</h2>

<p>Internist </p>

<p>200012</p>

<button onclick="toggleCard('card12')">Click to view details</button>

<div class="Additional-details">

<p>Phone:0772345678</p>

<p>Email: khalid.ahmad@clinic.com</p>

<p>Experience: 16 years</p>

<p>Work Hours: Sunday to Thursday, 8:00 AM - 4:00 PM</p>

</div>

</div>

<div class="doctor-card" id="card13">

<h2>Dr. Olaa Ahmad</h2>

<p>Cardiologist</p>

<p>200013</p>

<button onclick="toggleCard('card13')">Click to view details</button>

<div class="Additional-details">

<p>Phone:0773456789</p>

<p>Email: olah.ahmad@clinic.com</p>

<p>Experience: 10 years</p>

<p>Work Hours: Monday to Friday, 10:00 AM - 6:00 PM</p>

</div>

</div>

<div class="doctor-card" id="card14">

<h2>Dr. Sawsan Ameer</h2>

<p>Neurologist</p>

<p>200014</p>

<button onclick="toggleCard('card14')">Click to view details</button>

<div class="Additional-details">

<p>Phone:0774567890</p>

<p>Email: sawsan.ameer@clinic.com</p>

<p>Experience: 8 years</p>

<p>Work Hours: Sunday to Thursday, 8:30 AM - 4:30 PM</p>

</div>

</div>

<div class="doctor-card" id="card15">

<h2>Dr. Ameer Fouad</h2>

<p>Psychiatrist</p>

<p>200015</p>

<button onclick="toggleCard('card15')">Click to view details</button>

<div class="Additional-details">

<p>Phone:0775678901</p>

<p>Email: ameer.fouad@clinic.com</p>

<p>Experience: 18 years</p>

<p>Work Hours: Sunday to Thursday, 9:00 AM - 5:00 PM</p>

</div>

</div>

<div class="doctor-card" id="card16">

4">

```

<h2>Dr. Areen Ali</h2>
<p>Pulmonologist</p>
<p>200016</p>
<button onclick="toggleCard('card16')">Click to view details</button>
<div class="Additional-details">
  <p><strong>Phone:</strong>0796789012</p>
  <p><strong>Email:</strong> areen.ali@clinic.com</p>
  <p><strong>Experience:</strong> 5 years</p>
  <p><strong>Work Hours:</strong> Monday to Saturday, 9:30 AM - 5:30 PM</p>
</div>
</div>
<div class="doctor-card" id="card17">
  
  <h2>Dr. Mohammad Ibraheem</h2>
  <p>Orthopedic Surgeon</p>
  <p>200017</p>
  <button onclick="toggleCard('card17')">Click to view details</button>
  <div class="Additional-details">
    <p><strong>Phone:</strong>0786789012</p>
    <p><strong>Email:</strong> mohammad.ibraheem@clinic.com</p>
    <p><strong>Experience:</strong> 20 years</p>
    <p><strong>Work Hours:</strong> Sunday to Thursday, 8:00 AM - 4:00 PM</p>
  </div>
</div>
<div class="doctor-card" id="card18">
  
  <h2>Dr. Mahmood Esaam</h2>
  <p>Cardiothoracic Surgeon</p>
  <p>200018</p>
  <button onclick="toggleCard('card18')">Click to view details</button>
  <div class="Additional-details">
    <p><strong>Phone:</strong>0776789012</p>
    <p><strong>Email:</strong> mahmood.esaam@clinic.com</p>
    <p><strong>Experience:</strong> 22 years</p>
    <p><strong>Work Hours:</strong> Sunday to Thursday, 9:00 AM - 5:00 PM</p>
  </div>
</div>
<div class="doctor-card" id="card19">
  
  <h2>Dr. Rami Mohammad</h2>
  <p>ENT Surgeon </p>
  <p>200019</p>
  <button onclick="toggleCard('card19')">Click to view details</button>
  <div class="Additional-details">
    <p><strong>Phone:</strong>0797890123</p>
    <p><strong>Email:</strong> rami.mohammad@clinic.com</p>
    <p><strong>Experience:</strong> 12 years</p>
    <p><strong>Work Hours:</strong> Monday to Friday, 10:00 AM - 6:00 PM</p>
  </div>
</div>
<div class="doctor-card" id="card20">
  
  <h2>Dr. Mohammad Khaled</h2>

```

```

<p>Gynecologist</p>
<p>200020</p>
<button onclick="toggleCard('card20')">Click to view details</button>
<div class="Additional-details">
  <p><strong>Phone:</strong>0787890123</p>
  <p><strong>Email:</strong> mohammad.khaled@clinic.com</p>
  <p><strong>Experience:</strong> 8 years</p>
  <p><strong>Work Hours:</strong> Sunday to Thursday, 8:00 AM - 4:00 PM</p>
</div>
</div>
</div>
</body>
</html>

```

Code 14(Javascript for Doctors page)

```

function toggleCard(cardId) {
  const card = document.getElementById(cardId);
  const details = card.querySelector(".Additional-details");

  if (details.style.display === "none" || details.style.display === "") {
    details.style.display = "block";
  } else {
    details.style.display = "none";
  }
}

```

Code 15 (HTML for Analytics page)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <title>Analytics Dashboard - MedAnalyzer System</title>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <link rel="icon" href="https://png.pngfind.com/pngs/s/254-2548715_labor-analytics-icon-circle-hd-png-download.png">
  <link rel="stylesheet" href="{ { url_for('static', filename='Analytics.css') } }">
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></script>
  <script src="{ { url_for('static', filename='Analytics.js') } }" defer></script>
</head>
<body>
  <h2><img alt="magnifying glass icon" data-bbox="148 621 168 638"/> Patient Data Analytics</h2>

  <div class="filter-panel">
    <div class="filter-group">
      <label for="patientIdInput">Patient ID:</label>
      <input type="text" id="patientIdInput" placeholder="Enter Patient ID" name="patient_id" />
    </div>

    <div>
      <label>Measurements:</label>
      <div class="checkbox-group">
        <label><input type="checkbox" class="metric-checkbox" value="HeartRate"> Heart Rate</label>
        <label><input type="checkbox" class="metric-checkbox" value="BodyTemperature"> Body Temp</label>
        <label><input type="checkbox" class="metric-checkbox" value="OxygenSaturation"> Oxygen Sat</label>
        <label><input type="checkbox" class="metric-checkbox" value="SystolicBloodPressure"> Systolic BP</label>
        <label><input type="checkbox" class="metric-checkbox" value="DiastolicBloodPressure"> Diastolic BP</label>
        <label><input type="checkbox" onclick="selectAllMetrics(this)"> Select All</label>
      </div>
    </div>
  </div>
</body>
</html>

```

```

<div class="cards" id="resultsCards">
  <div class="card">❏ Avg. Heart Rate: --</div>
  <div class="card">📏 Avg. Temp: --</div>
  <div class="card">❏ Blood Pressure: --</div>
</div>

<div class="chart-container">
  <div id="chartsContainer"></div>
</div>

<!-- زر التنبؤ ونتيجته -->
<div style="margin-top: 20px;">
  <button onclick="predictRisk()" style="padding: 8px 16px; background-color:white; color: black; border: none; border-radius: 4px;">🔮 Predict Risk</button>
  <div id="predictionResult" style="margin-top: 10px; font-weight: bold;"></div>
</div>

<div class="notes-section">
  <label for="doctorNotes">📝 Doctor Notes:</label>
  <textarea id="doctorNotes" rows="5" cols="60" placeholder="Write notes here..."></textarea>
</div>

<div class="download-section">
  <button id="downloadReportBtn">❏ Download Report</button>
</div>
<script>
const userRole = "{{ role }}";
const patientIdFromServer = "{{ patient_id }}";
</script>

<script>
async function predictRisk() {
  const patientIdInput = document.getElementById('patientIdInput');
  const patientId = patientIdInput && patientIdInput.value.trim() ? patientIdInput.value.trim() : patientIdFromServer;

  if (!patientId) {
    document.getElementById("predictionResult").innerText = "🔮 Please enter a Patient ID.";
    return;
  }

  try {
    const response = await fetch('/predict_risk', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ patient_id: patientId })
    });

    const contentType = response.headers.get("content-type");

    if (!response.ok) {
      const errorText = await response.text();
      throw new Error(errorText);
    }

    if (!contentType || !contentType.includes("application/json")) {

```

```

    const errorText = await response.text();
    throw new Error("Expected JSON but received HTML: " + errorText.slice(0, 100));
  }

  const result = await response.json();
  document.getElementById("predictionResult").innerText = `🤖 Risk Prediction: ${result.risk_prediction}`;

} catch (error) {
  document.getElementById("predictionResult").innerText = `🚫 Server error: ${error.message}`;
}
}
</script>

```

</body>

</html>

Code 16 (javascript for Analytics page)

```

document.addEventListener('DOMContentLoaded', () => {
  const BASE_URL = 'http://127.0.0.1:5000';

  const patientIdInput = document.getElementById('patientIdInput');
  const predictionDiv = document.getElementById('predictionResult');
  const checkboxes = document.querySelectorAll('.metric-checkbox');
  const downloadBtn = document.getElementById('downloadReportBtn');
  const chartsContainer = document.getElementById('chartsContainer');

  // تحديد الكل
  window.selectAllMetrics = function (selectAllCheckbox) {
    const isChecked = selectAllCheckbox.checked;
    checkboxes.forEach(cb => { cb.checked = isChecked; });
    triggerFetchAndRender();
  };

  // تحميل التقرير PDF
  if (downloadBtn) {
    downloadBtn.addEventListener('click', () => {
      const { jsPDF } = window.jspdf;
      const doc = new jsPDF();
      doc.setFontSize(18);
      doc.text("Patient Report - MedAnalyzer", 20, 20);

      const notes = document.getElementById('doctorNotes')?.value || "";
      doc.setFontSize(12);
      doc.text("Doctor Notes:", 20, 35);
      doc.text(notes, 20, 45);

      const charts = document.querySelectorAll("#chartsContainer canvas");
      let yPosition = 60;

      for (let i = 0; i < charts.length; i++) {
        const chart = charts[i];
        const imgData = chart.toDataURL('image/png');
        if (yPosition > 250) {
          doc.addPage();
          yPosition = 20;
        }
        doc.addImage(imgData, 'PNG', 20, yPosition, 160, 90);
        yPosition += 100;
      }
    });
  }
}

```

```

    doc.save("patient_report.pdf");
  });
} else {
  console.error("The upload button is missing from the page ✖!");
}
if (typeof patientIdFromServer !== 'undefined' && patientIdFromServer) {
  patientIdInput.value = patientIdFromServer;
  triggerFetchAndRender();
}

patientIdInput.addEventListener('input', triggerFetchAndRender);
patientIdInput.addEventListener('change', triggerFetchAndRender);
checkboxes.forEach(cb => cb.addEventListener('change', triggerFetchAndRender));

function triggerFetchAndRender() {
  const patientId = patientIdInput.value.trim();
  if (!patientId) return;

  const selectedMetrics = Array.from(checkboxes)
    .filter(cb => cb.checked)
    .map(cb => cb.value);

  if (selectedMetrics.length === 0) return;

  console.log("🔍 Fetching for patient ID:", patientId);

  fetch(`${BASE_URL}/get_patient_data`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ patient_id: patientId, metrics: selectedMetrics }),
  })
    .then(res => res.json())
    .then(data => {
      if (data.error) {
        predictionDiv.innerHTML = `

${data.error}</p>`;
        return;
      }

      renderCharts(data);
      updateCards(data);

      fetch(`${BASE_URL}/predict_risk`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ patient_id: patientId }),
      })
        .then(res => res.json())
        .then(predictionData => {
          if (predictionData.error) {
            predictionDiv.innerHTML = `

${predictionData.error}</p>`;
          } else {
            predictionDiv.innerHTML = `
              <h3>📊 Risk Prediction:
              <span style="color:${predictionData.risk_prediction === "High Risk" ? "red" : "green"}">
                ${predictionData.risk_prediction}
              </span>
              </h3>
            `;
          }
        });
    });


```

```

    }
  })
  .catch(err => {
    predictionDiv.innerHTML = `

Prediction error: ${err.message}</p>`;
    console.error('Prediction fetch error:', err);
  });
})
.catch(err => {
  predictionDiv.innerHTML = `

Error: ${err.message}</p>`;
  console.error('Fetch error:', err);
});
}


```

```

function renderCharts(data) {
  chartsContainer.innerHTML = "";
  const timestamps = data.timestamps || [];

```

```

  Object.keys(data).forEach(metric => {
    if (!['timestamps', 'prediction', 'plot_url'].includes(metric)) return;

```

```

    const canvas = document.createElement('canvas');
    chartsContainer.appendChild(canvas);

```

```

    new Chart(canvas, {
      type: 'line',
      data: {
        labels: timestamps,
        datasets: [{
          label: metric,
          data: data[metric],
          borderColor: 'rgba(75, 192, 192, 1)',
          fill: false,
          tension: 0.1,
        }],
      },
      options: {
        responsive: true,
        plugins: {
          title: {
            display: true,
            text: `${metric} over time`,
          },
        },
      },
    });
  });
}

```

```

function updateCards(data) {
  const heartRates = data['HeartRate'] || [];
  const temperatures = data['BodyTemperature'] || [];
  const systolic = data['SystolicBloodPressure'] || [];
  const diastolic = data['DiastolicBloodPressure'] || [];

```

```

  const avg = arr => arr.length ? (arr.reduce((a, b) => a + b, 0) / arr.length).toFixed(1) : '--';

```

```

  const cards = document.querySelectorAll('.card');
  if (cards.length >= 3) {

```

```

cards[0].textContent = `□ Avg. Heart Rate: ${avg(heartRates)}`;
cards[1].textContent = `⚡ Avg. Temp: ${avg(temperatures)}`;
cards[2].textContent = `□ Blood Pressure: ${avg(systolic)}/${avg(diastolic)}`;
}
}

```

});

Code 17 (HTML fore About page)

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <title>About-MedAnalyzer System</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Abril+Fatface&family=Caveat:wght@400..700&family=Oswald:wght@200..700&display=swap" rel="stylesheet">
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link rel="icon" href="https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcRd-
UCUhSJeyIT1t9IZ_u6r5NID8T_I4He0bbXXidUBDnhEJ76PaSO1HKpC36fmapfOQBc&usqp=CAU">
    <link rel="stylesheet" href="{{ url_for('static', filename='about.css') }}">
    <!-- داخل قالب HTML (مثل forgot_password.html) -->
    <script src="{{ url_for('static', filename='main.js') }}" defer></script>
  </head>
  <body>
    <div class="container">
      <h1>About the Project</h1>
      <h2>Project Title</h2>
      <p>MedAnalyzer System</p>
      <h2>Project Idea / Overview</h2>
      <p>
        This system allows healthcare providers or users to upload patient data in CSV format, then analyzes key health
        indicators such as BMI, blood pressure, sugar levels, and more.
        It automatically detects critical health cases and generates visual graphs and downloadable PDF reports.
      </p>
      <h2>Project Goals</h2>
      <ul>
        <li>Analyze health data efficiently</li>
        <li>Automatically identify critical cases</li>
        <li>Visualize data in a user-friendly way</li>
        <li>Generate downloadable health reports</li>
      </ul>
      <h2>Key Features</h2>
      <ul>
        <li>BMI Calculation</li>
        <li>Blood Pressure Analysis</li>
        <li>Blood Sugar Monitoring</li>
        <li>Temperature & Heart Rate Tracking</li>
        <li>Smoking Status Analysis</li>
        <li>Gender & Lifestyle-based statistics</li>
        <li>Graph generation</li>
        <li>PDF Report Export</li>
      </ul>
      <h2>Technologies Used</h2>
      <ul>
        <li><strong>Backend:</strong> Python, Pandas, Matplotlib, Seaborn, FPDF, Flask</li>
        <li><strong>Frontend:</strong> HTML, CSS, JavaScript, Chart.js Library, jsPDF Library</li>
        <li><strong>Data Format:</strong> CSV</li>
      </ul>
    </div>
  </body>
</html>

```

```

        <li><strong>Communication:</strong> REST API between frontend and backend</li>
    </ul>
    <h2>Team Members</h2>
    <ul>
        <li>Amal Alawadi – UI/UX Designer & Frontend Developer</li>
        <li>Tamara Malkawi & Sora Mahasneh – Backend Developer and API Integrator</li>
        <li>Deema Alkhaldi – Machine learning</li>
    </ul>
    <h2>Additional Notes</h2>
    <p class="note">
        The system is designed for educational purposes and simulates a simplified version of real patient monitoring tools
        used in healthcare environments.
    </p>
</div>
</body>
</html>

```

Code 18 (Main JavaScript)

```

document.addEventListener("DOMContentLoaded", function () {

    const BASE_URL = 'http://127.0.0.1:5000';
    // toggle password و role change
    function togglePassword() {
        const passwordInput = document.getElementById("password");
        const toggleIcon = document.getElementById("togglePassword");
        const type = passwordInput.getAttribute("type") === "password" ? "text" : "password";
        passwordInput.setAttribute("type", type);
        toggleIcon.classList.toggle("fa-eye");
        toggleIcon.classList.toggle("fa-eye-slash");
    }

    function handleRoleChange() {
        const roleSelect = document.getElementById("role");
        const patientIdField = document.getElementById("patientIdField");

        if (roleSelect.value === "patient") {
            patientIdField.style.display = "block";
        } else {
            patientIdField.style.display = "none";
            document.getElementById("patient_id").value = "";
        }
    }

    // حسب الدور (تحديث عند تغير القيمة) patient ID إظهار أو إخفاء حقل
    const roleSelect = document.getElementById("role");
    const patientIdField = document.getElementById("patientIdField");
    roleSelect.addEventListener("change", function () {
        if (this.value === "patient") {
            patientIdField.style.display = "block";
        } else {
            patientIdField.style.display = "none";
        }
    });

    // معالجة النتائج + fetch إرسال نموذج تسجيل الدخول باستخدام
    const loginForm = document.getElementById("loginForm");
    const errorMsg = document.getElementById("login-error");

    if (loginForm) {

```

```

loginForm.addEventListener("submit", function (e) {
  e.preventDefault(); // منع الإرسال التقليدي

  const email = document.getElementById("email").value;
  const password = document.getElementById("password").value;
  const role = roleSelect.value;
  const patient_id = document.getElementById("patient_id").value;

  fetch("/login", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      email,
      password,
      role,
      patient_id: role === "patient" ? patient_id : null
    })
  })
  .then(response => response.json())
  .then(data => {
    if (data.success) {
      window.location.href = "/home"; // أو أي صفحة بعد تسجيل الدخول
    } else {
      errorMsg.style.display = "block";
      errorMsg.textContent = data.message || "خطأ في تسجيل الدخول.";
    }
  })
  .catch(err => {
    errorMsg.style.display = "block";
    errorMsg.textContent = "حدث خطأ في الاتصال بالخادم.";
    console.error("Login Error:", err);
  });
});
}

// loginForm إذا عندك نموذج - (FormData طريقة أخرى باستخدام) إرسال نموذج تسجيل الدخول
if (loginForm) {
  loginForm.addEventListener('submit', async function (e) {
    e.preventDefault();
    const formData = new FormData(loginForm);

    const res = await fetch(`${BASE_URL}/login`, {
      method: 'POST',
      body: formData,
      credentials: 'include'
    });

    const text = await res.text();
    if (res.redirected) {
      window.location.href = res.url;
    } else {
      const loginMessage = document.getElementById('loginMessage');
      if (loginMessage) loginMessage.textContent = text || 'Login failed.';
    }
  });
}

```

```

}

// بحث في input_with_icon
const searchInput = document.getElementById("input_with_icon");
if (searchInput) {
  searchInput.addEventListener("input", function () {
    const query = searchInput.value.toLowerCase();
    console.log("Searching for: " + query);
  });
}

// تسجيل الخروج
const authButton = document.getElementById("authBtn");
if (authButton) {
  authButton.addEventListener("click", function () {
    alert("You have successfully logged out");
    window.location.href = "Login.html";
  });
}

// تفاصيل الكارد Toggle
function toggleCard(cardId) {
  const card = document.getElementById(cardId);
  const details = card.querySelector('.Additional-details');
  if (details.style.display === 'block') {
    details.style.display = 'none';
  } else {
    details.style.display = 'block';
  }
}

console.log("About page loaded!");

// نوت تفاعلية
const note = document.querySelector('.note');
if (note) {
  note.addEventListener('click', () => {
    alert("This is for educational use only.");
  });
}

// حفظ بيانات المريض
const form = document.getElementById("patient-form");
const saveButton = document.querySelector(".btn-save");

if (form && saveButton) {
  saveButton.addEventListener("click", async function (event) {
    event.preventDefault();

    const formData = new FormData(form);

    const checkedHistories = [];
    const historyCheckboxes = document.querySelectorAll('input[name="patient_history"]:checked');

    historyCheckboxes.forEach(cb => {
      if (cb.value === "Other") {
        const otherText = document.getElementById("other_history_input")?.value.trim();
        if (otherText) {
          checkedHistories.push(otherText);
        }
      }
    });
  });
}

```

```

        } else {
            checkedHistories.push("Other");
        }
    } else {
        checkedHistories.push(cb.value);
    }
});

formData.delete("patient_history");
formData.append("patient_history", checkedHistories.join(", "));

if (!formData.get("Timestamp")) {
    const now = new Date();
    formData.set("Timestamp", now.toISOString());
}

try {
    const response = await fetch("/save_patient", {
        method: "POST",
        body: formData
    });

    if (response.ok) {
        alert("✓ Patient data saved successfully.");
        form.reset();
    } else {
        const errorText = await response.text();
        alert("✗ Failed to save data: " + errorText);
    }
} catch (error) {
    console.error("✗ Server communication error:", error);
    alert("✗ An error occurred while connecting to the server.");
}
});
}

// تسجيل الخروج
const logoutBtn = document.getElementById("logout-Btn") || document.querySelector(".logout-btn");

if (logoutBtn) {
    logoutBtn.addEventListener("click", async function (e) {
        e.preventDefault();

        try {
            const response = await fetch("/logout", {
                method: "POST",
                credentials: "same-origin",
                headers: {
                    "Content-Type": "application/json"
                }
            });

            if (response.ok) {
                window.location.href = "/login";
            } else {
                alert("✗ Logout failed.");
            }
        } catch (error) {

```

```

        console.error(" ✖ Logout error:", error);
        alert(" ✖ Could not connect to the server.");
    }
});
}

```

// القياسات المختارة PatientID رسم بياني للقياسات الزمنية حسب

```

const patientSelect = document.getElementById("patientSelect");
const checkboxes = document.querySelectorAll(".metric-checkbox");
const chartCanvas = document.getElementById("metricsChart");

```

```

async function fetchPatientData() {
    const patientId = patientSelect.value;
    if (!patientId) return null;

    const response = await fetch("/analytics_data", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({ patient_id: patientId })
    });

    if (!response.ok) {
        console.error("Failed to fetch data");
        return null;
    }

    const data = await response.json();
    return data;
}

```

```

async function updateChart() {
    const selectedMetrics = Array.from(checkboxes)
        .filter(cb => cb.checked)
        .map(cb => cb.value);

    if (!patientSelect.value || selectedMetrics.length === 0) {
        if (window.chartInstance) {
            window.chartInstance.destroy();
            window.chartInstance = null;
        }
        chartCanvas.getContext('2d').clearRect(0, 0, chartCanvas.width, chartCanvas.height);
        return;
    }
}

```

```

const data = await fetchPatientData();
if (!data || data.error) {
    alert(data?.error || "No data found.");
    if (window.chartInstance) {
        window.chartInstance.destroy();
        window.chartInstance = null;
    }
    return;
}

```

```

const labels = data.timestamps;
const datasets = selectedMetrics.map(metric => ({

```

```

        label: metric,
        data: data.values[metric] || [],
        borderColor: getRandomColor(),
        fill: false,
        tension: 0.3
    }));

    if (window.chartInstance) {
        window.chartInstance.destroy();
    }

    window.chartInstance = new Chart(chartCanvas, {
        type: "line",
        data: { labels, datasets },
        options: {
            responsive: true,
            scales: {
                x: {
                    title: { display: true, text: "Timestamp" }
                },
                y: {
                    beginAtZero: false
                }
            }
        }
    });
}

function getRandomColor() {
    const letters = "0123456789ABCDEF";
    let color = "#";
    for (let i = 0; i < 6; i++) {
        color += letters[Math.floor(Math.random() * 16)];
    }
    return color;
}

if (patientSelect) {
    patientSelect.addEventListener("change", updateChart);
}

if (checkboxes) {
    checkboxes.forEach(cb => cb.addEventListener("change", updateChart));
}
});
Code 19 (JavaScript for upload file )

```

```

function uploadfile() {
    const BASE_URL = 'http://127.0.0.1:5000';
    const fileInput = document.getElementById("file-upload");
    const file = fileInput.files[0];

```

```

    if (!file) {
        document.getElementById("warningMessage").innerText = "Please select a file before
uploading.";
        document.getElementById("warningMessage").style.display = "block";
        return;
    }

    const formData = new FormData();
    formData.append("file-upload", file); // هنا "file" لاحظ الاسم

    fetch("/upload", {
        method: "POST",
        body: formData
    })
    .then(response => response.json())
    .then(data => {
        alert(data.message);
    })
    .catch(error => {
        console.error("Error uploading file:", error);
    });
}

```

Code 20 prediction model

53s

```
import pandas as pd
from ydata_profiling import ProfileReport

# Load the dataset
df = pd.read_csv('/content/human_vital_signs_dataset_2024.csv')

# Create the profiling report
profile = ProfileReport(df,
                        title="Exploratory Data Analysis Report - Human Vital Signs Dataset",
                        explorative=True,
                        minimal=False,
                        progress_bar=True)

# Export report to HTML file
profile.to_file("eda_report.html")

print("EDA report has been successfully generated and saved as 'eda_report.html'")
```

```
[3] #Dropping columns:
cols_to_drop = ['Derived_HRV', 'Derived_Pulse_Pressure', 'Derived_MAP', 'Weight (kg)', 'Height (m)']
df.drop(columns=cols_to_drop, errors='ignore', inplace=True)

df.drop(columns=['Patient ID', 'Timestamp'], errors='ignore', inplace=True)

# Mapping 'Risk Category':
df['Risk Category'] = df['Risk Category'].map({'Low Risk': 0, 'High Risk': 1})

# Mapping 'Gender'
gender_mapping = {'Male': 0, 'Female': 1}
if 'Gender' in df.columns:
    df['Gender'] = df['Gender'].map(gender_mapping)
else:
    print("Warning: Column 'Gender' not found.")

# Handling missing values
df.dropna(subset=['Risk Category', 'Gender'], inplace=True)

df.dropna(inplace=True)

# Resetting index:
df.reset_index(drop=True, inplace=True)
```

```
0s [4] # number of rows (samples) in the DataFrame df after the data cleaning steps have been applied
print(f"Number of samples after cleaning: {df.shape[0]}")
```

Number of samples after cleaning: 65078

✓  # Splitting the data: It creates two new DataFrames:
X = df.drop(columns=['Risk Category'])
y = df['Risk Category']

✓  [6] # Applying StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

Saving the scaler:
joblib.dump(scaler, 'scaler.pkl')
print("Scaler saved as 'scaler.pkl'")

 Scaler saved as 'scaler.pkl'

✓   with open("feature_names.json", "w") as f:
 json.dump(list(X.columns), f)
 print("Feature names saved as 'feature_names.json'")

 Feature names saved as 'feature_names.json'

 # Cross-validation setup:
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_accuracies = []

for fold, (train_idx, val_idx) in enumerate(skf.split(X_scaled, y), 1):
 X_train_cv, X_val_cv = X_scaled[train_idx], X_scaled[val_idx]
 y_train_cv, y_val_cv = y.iloc[train_idx], y.iloc[val_idx]

 model_cv = xgb.XGBClassifier(
 use_label_encoder=False,
 eval_metric='logloss',
 max_depth=5,
 learning_rate=0.1,
 n_estimators=100
)

 model_cv.fit(X_train_cv, y_train_cv)
 y_pred_cv = model_cv.predict(X_val_cv)
 acc = accuracy_score(y_val_cv, y_pred_cv)
 cv_accuracies.append(acc)

 print(f"\nFold {fold} Accuracy: {acc:.4f}")
 print("Classification Report:")
 print(classification_report(y_val_cv, y_pred_cv))
 cm = confusion_matrix(y_val_cv, y_pred_cv)
 print("Confusion Matrix:\n", cm)

```

✓ 3s [9] # trains the final XGBoost
final_model = xgb.XGBClassifier(
    use_label_encoder=False,
    eval_metric='logloss',
    max_depth=5,
    learning_rate=0.1,
    n_estimators=100
)

final_model.fit(X_scaled, y)

```

```

✓ 0s import matplotlib.pyplot as plt

# Extracting feature names
with open("feature_names.json", "r") as f:
    feature_names = json.load(f)

# Extracting feature importances
importances = final_model.feature_importances_

# Creating and sorting DataFrame
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

# Printing importance table
print("\nFeature Importances:")
print(importance_df)

# Plotting importance
plt.figure(figsize=(10, 6))
plt.barh(importance_df['Feature'], importance_df['Importance'], color='skyblue')
plt.xlabel("Importance Score")
plt.title("Feature Importances from XGBoost")
plt.gca().invert_yaxis() # لعكس الترتيب بحيث الأعلى في الأعلى
plt.show()

```